



Экзамен ак

надо разобраться с флипфлоп! слишком частый вопрос мне кажется - это цифровое устройство, которое может переходить из одного устойчивого состояния в другое и попеременно менять состояния по мере поступления цифровых информационных сигналов. Говорим обычно о д-триггерах. Что-то с чтением данных. Работают быстрее потому что провода тормозят а это чисто электрическая схема

▼ 1) Что такое компьютерная система? Отличие информационной и управляющей систем? Почему большинство современных компьютерных систем считаются системами с преобладающей программной составляющей? Примеры. Понятие информационного процессора.

Компьютерная система - любая система, оснащённая внутренними алгоритмами управления

примеры: интерактивные детские игрушки, микроволновка (прикольно типа что реально сами по себе детали - нихуя не система, зато вместе оп-ля и супер гуд), умные часы

Информационная система - система, задача которой получить данные, обработать их и на этой основе выдать информацию (google, gpt, ворд, программки всякие которые транслируют файлы из одного формата в другой, хз что тут ещё). Приоритет - производительность (быстро быстро, супер быстро!) (но мне кажется это не так)

Управляющая система - система, задача которой взаимодействовать с реальным физическим миром, для управления и контроля над этой системой. Тут ваще супер важно время и плюс реактивность ответа и что

он всегда есть (те если какая-то аварийная ситуация что-то делаем). Тут примерчик Дашулька придумала: машины с автопилотом.

Разница - эксплуатация в разных целях, проектирование тоже разное

Почему большинство современных компьютерных систем считаются системами с преобладающей программной составляющей?

Системы с преобладающей программной составляющей - системы, в которых основным аспектом является разработка и/или интеграция программного обеспечения. Вот лично я думаю потому что это база. В прямом смысле этого слова. Типа это тот уровень, который действительно нужно серьёзно и обдуманно делать, потому что потом на него уже наслаиваются остальные части. Ну и плюс когда нам у какой-то системы захотелось что-то поменять, мы же не будем менять сами детальки в надежде что это будет дешевле и быстрее, а скорее поменяем отклик ПО на ту или иную задачу. Типа как с игрушкой какой-то. Ну не нравится нам что машинка едет вперед, а не назад. Проще перепрограммировать чем менять деталь, а главное кмк дешевле)

Информационный процессор - это система, принимающая информацию и обрабатывающая её посредством алгоритмов. Тут 4 основные части: ввод, процессор, память, вывод. Примерчик: микросервис по сбору просмотренных фильмов. Вы поставили оценку 10 на фильм "Судья", оценка пришла обработалась сохранилась. Куда-то выдалась инфа, что теперь оценка у "Судьи" 9.99)

В компухтере это например цпу) Всё пака лицо рука

▼ 2) Системная инженерия. Понятие системы. Варианты рассмотрения систем. Модульность. Жизненный цикл. Операционное окружение и

**обеспечивающие системы. Заинтересованные стороны (stakeholders).
Проблема передачи информации при разработке компьютерных систем.**

Системная инженерия - область научно-технической деятельности, основная задача которой - разработка успешной системы, которая в свою очередь удовлетворяет заинтересованные стороны (государство, заказчик и тд). Продумать нужно всё, начиная от концепции, заканчивая этапом выхода из эксплуатации.

Система - совокупность частей, обладающих единым смыслом и единой функцией.

Рассмотрение систем:

Система - совокупность частей, которые вместе дают новые свойства
Система - часть операционного окружения в которое она встроена, а также предъявляемые к ней требования

Модульность - свойство системы, при котором она разделена на отдельные независимые модули. Причём каждый из которых выполняет определённую функцию. Важно, что система обладающая этим свойством может быть декомпозирована (разделение целого на части). Модули системы обычно разработаны так, чтобы они были изолированы друг от друга, чтобы при случае было легче их заменить или обновить без влияния на основную систему.

Жизненный цикл:

- 1) Концепция (анализ требований и архитектурное представление)
- 2) Разработка (написание исходного кода или организация вычислительного процесса)
- 3) Производство (разработка аппаратного обеспечения, подготовка артефактов, в случае ПО - включает в себя деплой)
- 4) Использование

- 5) Поддержка (сидим, глядим, что всё хорошо)
- 6) Вывод из эксплуатации

Операционное окружение - среда, в которой работает операционная система (программное обеспечение, которое управляет компьютером и позволяет запускать на нём прикладные программы). Включает в себя всё, что используется операционной системой для выполнения её функций. (виртуальное адресное пространство, подсистема ввода-вывода, процессоры, память, устройства)

Обеспечивающие системы - то, что помогает двигать проекты между стадиями жизненного цикла

Стейкхолдер - заинтересованная сторона или участник процесса. По сути это лица, которые прямо или косвенно влияют на на компанию при этом ожидают определённых результатов деятельности.

Проблемы передачи информации при разработке компьютерных систем

- Различия в терминологии. Разработчики, заказчики и пользователи
- Различия в уровне знаний. Разный уровень знаний о системе.
- Различия в интересах. разные интересы в отношении системы.

▼ 3) Цели и задачи архитектурного проектирования компьютерных систем, его эффект. Понятие архитектуры. Различные трактовки (Гради Буч, ISO 42010, ...) и их практическая значимость

Цели и задачи архитектурного проектирования компьютерных систем:

Архитектурные ошибки такие знаете масштабные могут быть полноценно обнаружены только уже в процессе эксплуатации, что как вы понимаете ну не оч хорошо для бизнеса. Соответственно отсюда цели и задачи:

- Не допускать таких ошибок (это в идеале всё конечно)
- архитектура - это база, соответственно туда нужно заложить всё самое необходимое, чтобы в какой-то момент так называемый дом не поехал от отсутствия какого-то кирпичика. В то же время всё на свете в фундамент класть не надо, а то уйдём под землю)
- при хорошей архитектуре система работает быстрее и предсказуемо
- при качественной архитектуре проще поддерживать систему
- снижение стоимости
- проще масштабировать

Эффект: кажется можно пожертвовать временем и сидеть продумывать архитектуру, а можно сделать супер быстро зато может недолгосрочно. Короче время важно, надо думать! нужно оптимально

По Бучу:

архитектура - концепция, которая подразумевает, что лучший способ создать ПО - это создавать его по мере необходимости (чаще всего прогеры этим занимаются в процессе), а не всё сразу.

Подразумеваем, что вместо того чтобы строить всё и сразу, строим только функции, которые действительно необходимы в данный момент.

Плюс - более адаптивная штука получается, плюс сокращаем время и ресурсы. Однако минус какой? нужно постоянно взаимодействовать с пользователями...{

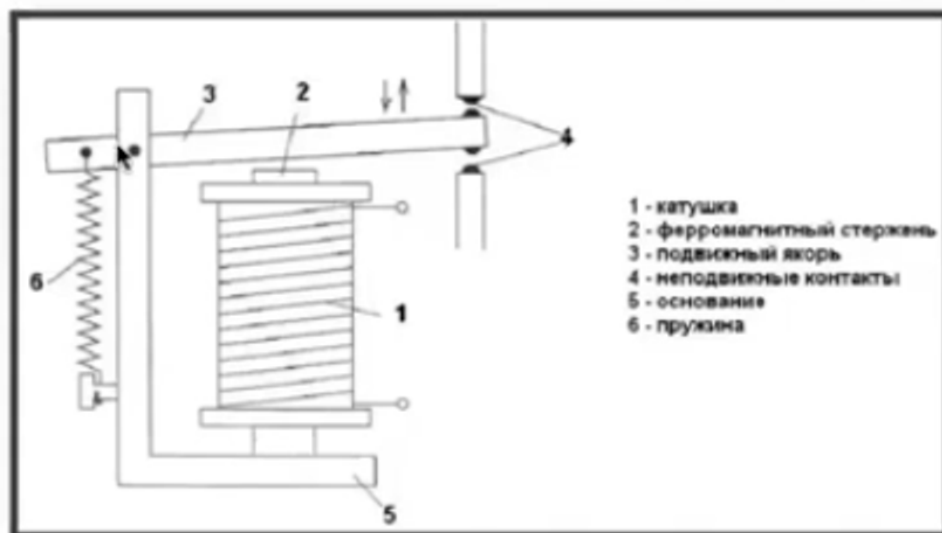
ISO 42010

архитектура. Тут значит направлено на стандартизацию всё. Стандартные термины, стандартизирована вся разработка. Этот стандарт помогает обеспечить ясность и согласованность в описании структуры, что облегчает понимание и анализ системы между разработчиками. Явно видим ключевые аспекты, на которых необходимо сконцентрироваться, отсюда идёт экономия времени. Да ещё прикольный поинт: для новичков удобно, так как всё стандартизировано, проще разобраться. Но тут явно видно, что ни о какой гибкости и речи нет, отсюда вот такой минус)

▼ **4) Реле как базис компьютерной системы. Область применения и принципы построения систем на базе реле. Примеры релейных схем и принцип их функционирования. Формирование логического базиса**

Устройство это используется для включения/выключения цепей под действием тока. В контексте компьютерных систем, реле можно использовать как элементарный базис для построения логических схем

Реле вот такая штука дрюка



Когда появляется ток на магнитной катушке, подвижный якорь примагничивается и цепь замыкается. Электрическая вещь, имитирующая ключ.

Классификации реле:

- нормально сомкнутое

- нормально разомкнутое

Область применения и принципы построения систем на базе реле

практика применения реле

- силовые шкафы управления
- там где электроника не работает, например, марсоходы
- программируемые логические контроллеры (ПЛК) для автоматизации производственных процессов (АСУТП)

принципы построения систем на базе реле

1. Использование Электромеханических Реле:

- Реле работает как простой электромеханический переключатель, активируемый электрическим током.

2. Логические Схемы:

- С помощью реле можно построить базовые логические элементы (И, ИЛИ, НЕ), которые могут быть соединены для создания сложных логических функций.

3. Отклик на Входные Сигналы:

- Система настроена таким образом, чтобы реагировать на входные сигналы (например, от датчиков) активацией или деактивацией определенных реле.

4. Последовательное и Параллельное Соединение:

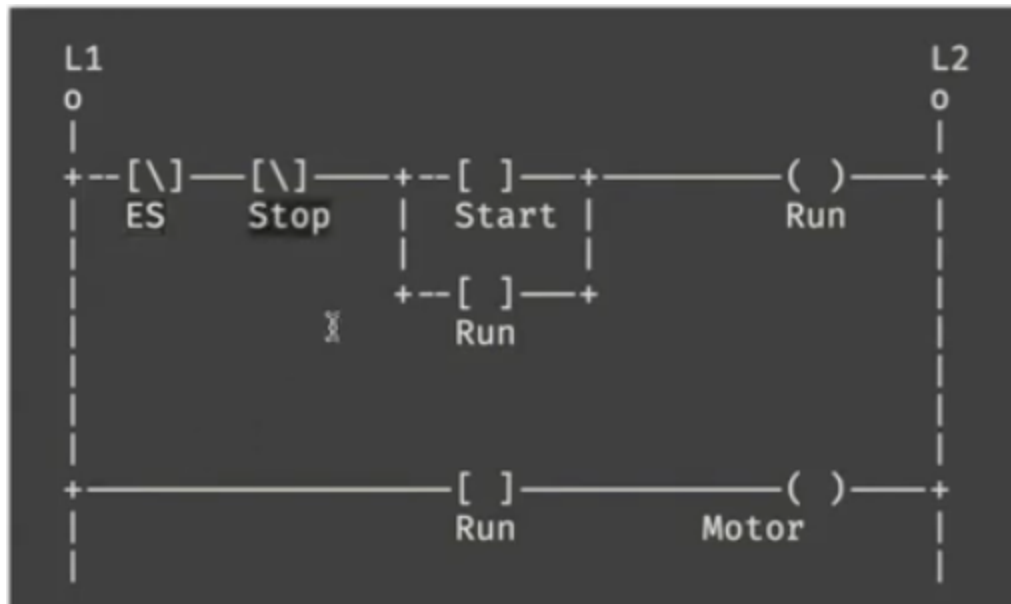
- Реле могут быть соединены последовательно или параллельно для выполнения сложных задач управления и логики.

5. Время Отклика и Задержки:

- Необходимо учитывать время реакции реле, которое может варьироваться и оказывать влияние на работу системы.

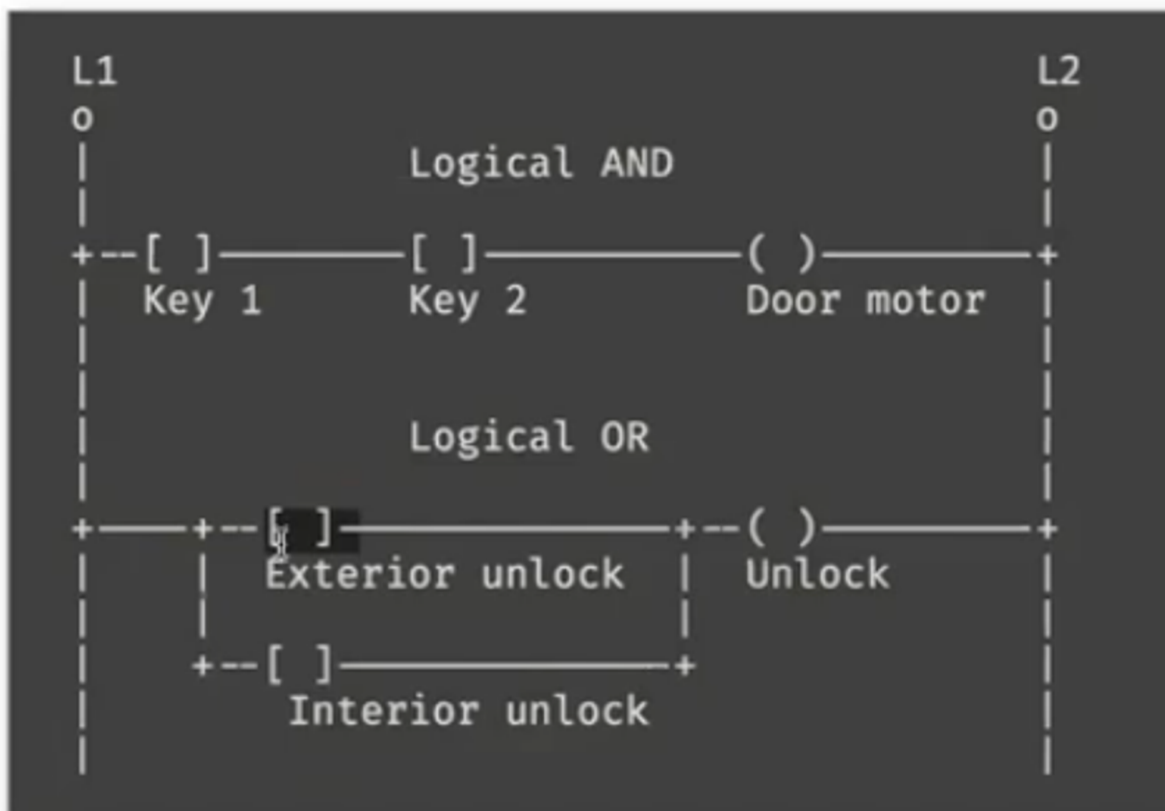
примеры релейных схем

классическая **схема включения электродвигателя** с аварийной остановкой



кнопка старт запустит мотор. кнопка ран будет поддерживать мотор в рабочем состоянии.

и, или



Формирование логического базиса

Формирование логического базиса - это процесс создания основных логических элементов, которые можно использовать для построения более сложных цифровых схем и систем. Логический базис обычно включает в себя несколько фундаментальных логических вентилей, которые представляют базовые булевы операции

Основные Логические Вентили

1. **И (AND):** Вентиль И выдает высокий уровень сигнала (1) только тогда, когда все его входы находятся на высоком уровне.
2. **ИЛИ (OR):** Вентиль ИЛИ выдает высокий уровень сигнала, если хотя бы один из его входов находится на высоком уровне.
3. **НЕ (NOT):** Вентиль НЕ инвертирует уровень сигнала на своем входе; если на входе высокий уровень сигнала, на выходе будет низкий, и наоборот.

▼ 5) Принципы кодирования информации в компьютерных системах.
Двоичный код, достоинства и недостатки. Машинное слово и адресация.
Код грея, BCD, Base64

Принципы кодирования информации в компьютерных системах:

1. двоичное кодирование
2. код грея
3. bcd
4. base64 и др

Сигнал передаётся по сети, таким образом тут он аналоговый. Потом пришёл на компухтер и стал цифровым, вот 4 способа описаны значит) потому что с таким понятно проще компухтеру работать

Двоичный код, достоинства и недостатки

двоичный код - способ представления информации в виде кода, в котором каждый разряд принимает одно из двух значений: 0 или 1 (обычно)

достоинства:

- простота в реализации (два возможных состояния, а не 10)
- простая арифметика
- надёжно и помехоустойчиво
- погрешность закладывается инженерами, а не определяется на этапе производства кабелей и тд
- многообразие информации - можем представить текст, числа, видосы и тырым пырым

недостатки:

- большой размер для больших чисел: числа, записанные в двоичной системе счисления могут занимать сильно много разрядов

- быстрый рост разрядов
- нативно человеком плохо читаемый набор нулей и единичек
- десятичные дроби плохо записываются (в виде бесконечных)

Машинное слово

машинное слово - это набор символов, хранящийся в оперативном запоминающем устройстве и воспринимаемый при обработке устройством машины, как единая группа. Машинное слово служит единицей информации.

представления данных:

есть big-endian (старший байт хранится первым), а есть little-endian

целые числа:

- позиционные сс (непозиционная: I, II, III, IV - римская)
- дополнительный код

дробные:

- с фиксированной точкой
- с плавающей точкой

Адресация

способ определения местоположения данных в памяти компьютере

Адресная шина: Ширина адресной шины - максимальный объём адресуемой памяти.

Типы адресации:

- Прямая адресация: каждая ячейка памяти имеет уникальный адрес, по которому инфу из неё можно дернуть. Инструкция обращается напрямую к этому адресу
- Косвенная адресация: адрес ячейки памяти хранится внутри другой ячейки памяти

- Индексная адресация: к адресу добавляется значение индексного регистра, что позволяет более эффективно работать с массивами и последовательностями данных
- Относительная адресация: вычисляется путём добавления к текущему адресу смещения - уменьшает объем памяти

код Грея

если есть варик высокой погрешности обычно используют. две соседние кодовые комбинации различаются только одним разрядом

Decimal	Gray Code	Binary
0	0000	0000
1	0001	0001
2	0011	0010
3	0010	0011
4	0110	0100
5	0111	0101
6	0101	0110
7	0100	0111
8	1100	1000
9	1101	1001
10	1111	1010
11	1110	1011
12	1010	1100
13	1011	1101
14	1001	1110
15	1000	1111

BCD (binary code decimal)

форма записи рациональных чисел, когда каждый десятичный разряд числа записывается в виде его четырёхбитного двоичных кода. Не нужны преобразования для вывода, но компактность теряется

Bit-Nr.	4	3	2	1
Wertigkeit	2^3 8	2^2 4	2^1 2	2^0 1
Dezimalziffern	0			
1				
2				
3				
4				
5				
6				
7				
8				
9				

Base64

стандарт кодирования двоичных данных при помощи только 64 символов ASCII. Алфавит кодирования содержит латинские символы A-Z, a-z, цифры 0-9 (всего 62 знака) и 2 дополнительных символа. Все пошло от HTTP и вопроса, зачем мне передавать биты, когда я могу передавать уже шифрованные слова. Достаточно компактно

▼ 6) Что такое комбинационная схема? Состояние и параллелизм в комбинационных схемах и схемах с регистрами, переходные процессы. Чтение и запись значений в регистры. Состояние x и z. Параллелизм

уровня битов. Особенности поведения комбинационных схем по сравнению с программами.

комбинационная схема - схема, составленная из набора логических элементов, в совокупности реализующая заданную таблицу истинности там есть И, ИЛИ, НЕ - элементы

Свойства комбинационной схемы:

1. Приход в стабильное состояние. Т.е. если мы подали на вход сигнал (либо 0 либо 1), значит у нас на выходе через какое-то точное время придёт на выход ответ (опять же либо 0, либо 1)
2. параллелизм уровня бит - узлы схемы работают параллельно
3. накопление ошибки в физическом смысле, что может привести к ошибке на логическом уровне. (каждая линия и логический элемент могут вносить погрешность, поэтому где-то должны стоять элементы, которые восстанавливают уровень сигнала на ожидаемый)

Состояния сигналов (про x и z):

0 и 1 - ну понятно база (при этом важно заметить что 0 это не напряжение равно 0, там есть какое-то небольшое, обычно примерно 0.3 Вольт)

x - неопределено, обычно используется в логических схемах, когда сигнал неопределён или неизвестен. Например, при инициализации схемы или при неправильной работе устройства.

z - выключено. Используется когда сигнал отключен от шины данных.

Состояние в комбинационной схеме:

комбинационные схемы не хранят состояние, однако иногда может потребоваться это делать. Чтобы хранить состояние используют триггеры
зачем нам нужно состояние?

1) сохранить какое-то значение (ск пользователь нажал на кнопку например)

- 2) сохранить состояние при двух параллельных тактовых генераторах (один работает быстрее, второй медленнее и чтоб один мог ждать второго нужно сохранить его результат
- 3) бонусный эффект от использования триггеров: выступает в роли буфера и сглаживает накопленную погрешность
- 4) программное управление - позволяет нам не дожидаться полного выполнения схемы, а вот самая левая оттарабанит и можно подавать новые значения на шины и в компараторы. а эти перейдут дальше и их значения не потеряются благодаря регистрам.

Триггер - название всех устройств, способных хранить состояние
самые распространённые

- d-триггер (delatyed)
подаётся сигнал и в триггере защёлкивается то значение, которое в нём было - буфер
- rs-триггер - выдаёт сигнал какой-то исходя из того что приходит

Таблица 1. Таблица истинности RS триггера.

R	S	Q(t)	Q(t+1)	Пояснения
0	0	0	0	Режим хранения информации R=S=0
0	0	1	1	
0	1	0	1	Режим установки единицы S=1
0	1	1	1	
1	0	0	0	Режим записи нуля R=1
1	0	1	0	
1	1	0	*	R=S=1 запрещенная комбинация
1	1	1	*	

условие изменения состояния:

- по фронту - получили 1 - защёлкнули
- по уровню - триггеры защёлкнуты только когда есть сигнал 1

Чтение и запись значений в регистры

Регистр - небольшие области памяти, которые могут быть прочитаны (по сути набор упорядоченных триггеров)

Чтение значений из регистров:

Чтение значений из регистров обычно выполняется с помощью специальной инструкции чтения, которая копирует значение из регистра в другое место в памяти. Это может быть регистр данных, регистр состояния или другое место в памяти. Инструкция чтения обычно указывает адрес регистра, из которого нужно прочитать данные 2.

Запись значений в регистры:

Запись значений в регистры обычно выполняется с помощью специальной инструкции записи, которая копирует значение из другого места в памяти в регистр. Это может быть регистр данных, регистр состояния или другое место в памяти. Инструкция записи обычно указывает адрес регистра, в который нужно записать данные, а также значение, которое нужно записать

Параллелизм и переходные процессы

в комб. схемамх

1. **Параллелизм:** В комбинационных схемах все элементы могут работать параллельно. Например, в сложной комбинационной схеме, состоящей из множества вентилях, каждый вентиль обрабатывает входные данные независимо и одновременно с другими.
2. **Переходные Процессы:** Переходные процессы в комбинационных схемах происходят при изменении входных сигналов. Задержка распространения сигнала через вентили определяет, как быстро выходные сигналы реагируют на изменение входных.

в схемах с регистрами

1. **Параллелизм:** Параллелизм в последовательностных схемах может быть менее явным, поскольку состояние схемы обновляется в определенные моменты времени (например, с каждым тактом часов).

Однако внутри самих регистров и других компонентов могут происходить параллельные процессы.

2. **Переходные Процессы:** Переходные процессы в последовательностных схемах сложнее, так как они включают изменение состояния памяти. Эти процессы часто синхронизируются с часовыми импульсами, что позволяет контролировать моменты обновления состояния.

особенности комб схем по сравнению с программами

1. все процессы между регистрами всегда происходят параллельно
2. нет понятия "система остановилась". система работает, пока есть питание
3. передача сигнала – физический аналоговый процесс. есть питание и контакт – есть передача
4. таблица истинности неполна – результат будет случайным

▼ **7) Аппаратное обеспечение. Виды монтажа и особенности производства. Навесной и поверхностный монтаж. Интегральные схемы. Проблемы специфичные для аппаратного обеспечения: производство, эксплуатация, устаревание.**

Аппаратное обеспечение - электронные и механические части вычислительного устройства. То это компьютерная система, если исключить из неё программное обеспечение и данные.

Виды монтажа:

два вида монтажа электронных схем

Навесной монтаж

Способ монтажа, при котором радиоэлементы соединяются друг с другом проводами или непосредственно выводами.

Плюсы:

- можно устанавливать или заменять элементы без вмешательства в конструкцию самого устройства
- скорость замены - не нужно опять же лезть в конструкцию

Минусы:

- сложность в автоматизации (на конвейер такую штуку не засунуть). В настоящем применяется редко и обычно только при монтаже крупногабаритных деталей

Поверхностный монтаж

технология, при которой электронные компоненты устанавливаются непосредственно на поверхность печатных плат, не в отверстия на них. Там может быть несколько слоёв

Плюсы:

- позволяет уменьшить размеры и вес устройства
- увеличить производительность и скорость монтажа
- можно автоматизировать
- всё прочно, маленько и круто!

Минусы:

- более сложное и дорогое оборудование
- высокие требования к качеству материалов и их хранения

Интегральные схемы

Интегральные схемы, также известные как микросхемы или чипы, представляют собой компактные электронные устройства, которые содержат все электронные компоненты и их взаимосвязи на одной полупроводниковой основе. Они изготовлены из кремния и обычно помещаются в неразборный корпус

Интегральные схемы нужны для усиления, манипуляции сигналами, логических процедур или хранения данных. Они могут содержать

транзисторы, резисторы, конденсаторы, диоды и другие электронные компоненты

Одним из главных преимуществ интегральных схем является их способность объединять на одном кристалле множество электронных компонентов и их взаимосвязей. Это позволяет значительно уменьшить размеры, массу и энергопотребление электронных устройств.

недостатки:

- могут быть сложными в производстве и требовать большого количества энергии для работы.
- Кроме того, они могут быть подвержены физическим повреждениям, таким как удары или высокие температуры, которые могут привести к их выходу из строя

В контексте производства интегральных схем нельзя не упомянуть закон Мура, согласно которому (в современной формулировке) количество транзисторов, размещаемых на кристалле интегральной схемы, удваивается каждые 24 месяца. Сдерживается следующими факторами:

- закон Амдала (фундаментальное ограничение на параллелизм, к нему мы вернемся позднее)
- объективная сложность параллельного программирования (современное многопоточное программирование может быть реализовано средним современным программистом только за счет большого количества абстракций, позволяющих просто работать с простыми случаями разработки, к этому вопросу мы вернемся позднее)
- тепловыделение и питание
- ограничения, связанные с физическими процессами (к примеру, когда не остается атомов для того, чтобы сложить затвор транзистора)

беды аппаратного обеспечения:

1) производство: дорого и сложно. Построить станки, сделать чистые зоны, создать заготовки и тд. Кроме того сюда же входят все трудности с

логистикой, складами, специалистами и гарантийными случаями

2) производство: задача при производстве аппаратного обеспечения: вместить как можно больше элементов (реле, конденсаторы, резисторы и всякого такого) на наименьшую площадь. Чтобы это было и эффективненько и дешево, ну и всем в нашем мире как будто, чем меньше, тем лучше

3) эксплуатация: оборудование может выходить из строя, для его починки нам необходим физический доступ, при проектировании компонентов мы должны учитывать среду в которой они будут использоваться. А в долгосрочке такие элементы подвержены устареванию: забивается пыль, а на олове, например, растут нитевидные кристаллы, которые могут замыкать схему

4) устаревание: могут быть несовместимы с новыми системами и устройствами, что может привести к невозможности передачи данных или работе с новыми программами. Устаревшие технологии могут быть менее эффективными и медленными по сравнению с современными аналогами. Кроме того, поддержка устаревших технологий может стать проблемой, так как разработчики и производители могут перестать выпускать обновления и запчасти для таких устройств

▼ 8) 2-этапное производство, варианты обеспечения. Понятия Hardware и Software, их свойства. Сравнение с понятиями программного и аппаратного обеспечения. Особенности. Причины разделения. Принципы совместного проектирования (HW/SW CoDesign)

Раньше аппаратура была не гибкой, из-за чего при малейших изменениях весь компьютер приходилось выкидывать. Тогда решили, что стоит унифицировать производство железа.

2-этапное производство заключается в идее разделить весь компьютер на hardware и software. hard - построить универсальный компьютер, а потом soft - написать под него ПО, которое в свою очередь подстраивается под определённую задачу.

понятия hardware and software:

под хардом понимается аппаратное обеспечение - физические компоненты компьютерной системы: процессор, память, жесткий диск. Это компоненты, которые выполняют основные функции такие как обработка данных и выполнение инструкций

под софтом - программное обеспечение - операционные системы, приложения, драйверы и другие программы, позволяющие пользователям взаимодействовать с хардом

на самом деле сейчас как говорят знающие люди хардвере и софтвере это не совсем АО и ПО, хард - система, которую сложно поменять в компе, а вот софт - то шо легко

Но легко поменять ≠ легко устроено. Ярким примером почему эти понятия нельзя связывать между собой является пример с ОС minix, которая стоит на большинстве интеловских процов. То есть по сути это ОС, это какой-то код, который вроде как относится к ПО, но в то же время это все лежит на железде и поменять ее функционал на ходу быстро и дешево очень тяжело, поэтому это все же относится к Hardware.

Свойства программного обеспечения

1. Быстрый цикл разработки.
2. Легко заменяется прямо у пользователя.
3. Пользователь как Beta-тестер.
4. Возможно удалённое обновление, в том числе без информированного согласия.
5. Сервис — вершина владения компьютерной системой.
6. Процесс создания и внедрения ПО автоматизируется (CI/CD).
7. Высокая сложность программ.

CoDesign - параллельная разработка системы. Сначала две команды договариваются об интерфейсах, системе команд, контракте, назвать можно как угодно. Затем они уходят параллельно делать HW и SW при этом опираясь на заранее обговоренные модели. После разработки происходит интеграция HW и SW друг с другом и фиксируются проблемы.

▼ **9) понятие модели вычислений (MoC). Сопоставление понятия парадигмы программирования и MoC. Примеры MoC (последовательные, параллельные, функциональные) и их роль в разработке компьютерных систем. Использование MoC в различных вычислительных платформах.**

MoC (model of computation) - модель вычислений - определяет каким будет вычислительный процесс для той или иной задачи. Определяет как устроен вычислитель и какие задачи он может решать. Это скорее теоретическое понятие. Нам важно это рассмотреть как систему команд процессора

- 1) МоК определяет возможности вычислительной машины
- 2) характеризует как как выполняется программа, какие состояния вычислителя, их последовательность, правила перехода
- 3) предоставляет язык для описания моделей процессов

сопоставление понятий:

парадигма это набор правил, который говорит программисту как надо программировать, в каком стиле и тд

А МоС говорит о том, как система(вот прям в общем смысле этого слова) работает/считает что-то

примеры МоК:

Последовательные - описывают процесс очередной смены состояний. В основном это все современные процессоры. Все инструкции друг за другом.

Функциональные модели – представляет собой вычислительный процесс в символьной форме и набор правил для переходов и сокращений

(комбинаторная логика)

Параллельные - у нас есть множество независимых, параллельно работающих CPU, которые по каким-то принципам взаимодействуют между собой (таймер, сообщение и тд).

использование мок

На практике модель вычислений используется в

1. Информатика:

- **Формальные модели:** Основа для создания и анализа алгоритмов и систем.

2. Дизайн языков программирования:

- Определяет синтаксис и семантику языков, что влияет на эффективность и удобство программирования.

3. Управление сложностью:

- Включает методы декомпозиции и абстракции, облегчая работу с большими и сложными системами.
- задаем себе жесткие рамки и получаем простую и предсказуемую программу

4. Ограничение творческого порыва:

- Сбалансированное внедрение новшеств, чтобы не усложнять систему без необходимости.

5. Модель-ориентированная инженерия:

- Использование моделей для улучшения разработки, включая xtUML, Capella.
- **Для ответственных приложений:** Обеспечивает четкую ответственность за каждый компонент системы.
- **Для переносимости:** Улучшает возможность перемещения компонентов между разными системами.

- **Для экспертизы:** Формализация знаний в области для лучшего понимания и передачи.

▼ 10) Универсальный процессор и его свойства. Машина Тьюринга и полнота по Тьюрингу. Виды процессоров (СБИС, FPGA, CGRA, CPU, GPU, DSP) и их сопоставление с точки зрения универсальности и эффективности

Для начала, обсудим понятие информационного процессора: информационный процессор - устройство, которое получает поток информации. Этот поток он преобразует, дополняет, фильтрует и возвращает. Что тут есть:

- вход
- выход
- механизмы обработки данных (процессор)
- хранилище данных (долгосрочное хранилище и оперативное хранилище(регистры))

Если мы хотим, чтобы наш информационный процессор стал универсальным, нужно:

- а) он должен решать высокий круг задач (возможность настройки конфигурации, что как раз позволит его использовать под разные фичи)
- б) мы должны иметь задать эту конфигурацию в зависимости от этапа производства

Примечание:

1. Универсальность $\neq$ полнота по Тьюрингу (полнота по Тьюрингу: значит, что алгоритм может быть вычислен на машине Тьюринга)
2. Теоретическая универсальность $\neq$ практическая (если мы на машине Тью можем посчитать любой алгоритм, это не значит что мы будем это делать на практике), когда мы говорим о реальных процессорах у нас есть какой-то круг задач под них. (использование графических

процессоров для нейронных сетей, нейронку посчитать на CPU можно конечно, но это будет долго пздц, а на графическом процессоре это будет шустро, при том что граф.проц. не шибко функциональный.)

3. Универсальность всегда противоречит эффективности. (у нас всегда есть трейд-офф между тем что мы будем делать решение подзадачи или же мы сделаем решение более гибким. (швейцарские ножи на 60,100 элементов - это звучит и выглядит красиво но по сути пользоваться этим не очень.)
4. Современный процессор - это не совсем то, что мы рассматриваем (мы рассматриваем самые простые урезанные), потому что у современных процессоров безумное количество программных и аппаратных обеспечений.

Свойства:

1. 2-этапное производство
2. Полнота по Тьюрингу
3. Отсутствие "серьезных" ограничений на "объем" программы.
(потенциально на машине тьюринга мы можем реализовать любой алгоритм, но главное чтобы ленты хватило)
4. изменяемость ПО (нет интереса говорить об универсальности процессора если он не перепрограммируемый, по хорошему мы должны иметь возможность менять ПО в рантайме и оно полностью отделено)

Машина Тьюринга

Машина Тьюринга - это абстрактная вычислительная модель, которая обладает свойством полноты по Тьюрингу. Это означает, что любую вычислительную задачу, которую можно решить с помощью любой другой вычислительной модели, можно также решить с помощью машины Тьюринга.

У Машины Тьюринга есть головка, которая может перемещаться по бесконечной магнитной ленте влево вправо , она может читать данные с ленты, записывать данные с ленты и принимать решения о том что делать с этими данными в зависимости от того какие данные считаны. Машина начинает с начального состояния и переходит в другие состояния согласно таблице переходов.

Он теоретически может выразить все задачи, решаемые компьютерами; почти все языки программирования Тьюринг-полные, если не обращать внимания на ограничения памяти.

Другими словами, можно написать на Java или C++ программу, которая будет действовать как машина Тьюринга. «Полными» также являются и HTML5, и CSS3. А вот SQL не полный — на нём пишутся в основном запросы к базе данных, а не полноценные программы.

Виды процессоров:

Стоит упомянуть что не существует идеальных процессоров, каждый из них это всё равно выбор между эффективностью и универсальностью и что-то ставится в приоритет в ущерб другому.

- чем больше на единицу вват мы можем вычислить операций - тем эффективнее будет система (для расчета биткойнов делали специализированные чипы)

крайности:

ASIC (СБИС) :

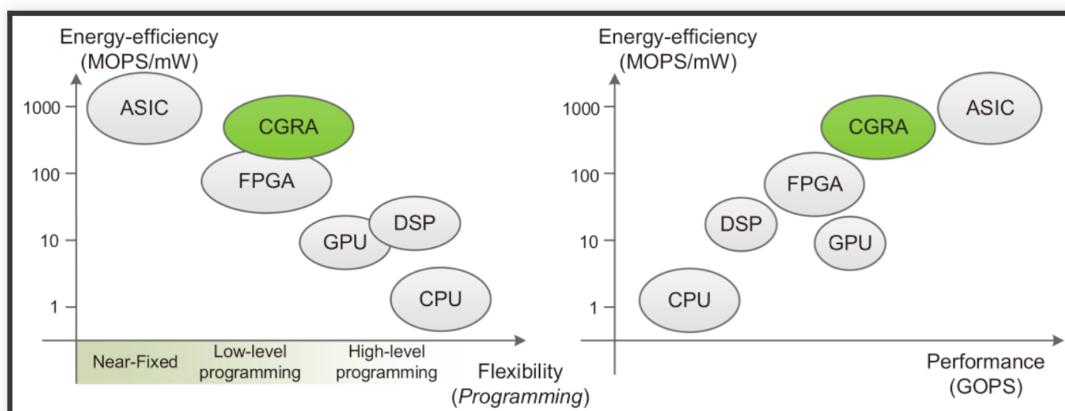
это специальная штука , которая только и могут что считать хэш-суммы для битка, но при этом реализовано максимально оптимизировано , все в кремнии , засчет этого потребляет мало электичества но супер быстро все считает.

CPU:

процессор общего назначения, который может решать любые алгоритмы (потенциально), но делает это медленно, неэффективно

промежуточные варианты:

- **CGRA** (конфигурируемая решетка графических элементов) - процессоры, специализированные для выполнения графических задач. Они обладают высокой производительностью в области графики, но могут быть менее универсальными в других областях.
- **FPGA** (программируемые матрицы вентилей) - процессоры, которые используют программируемую аппаратуру для выполнения задач. Они обладают высокой гибкостью и могут быть настроены для различных приложений, но могут быть менее эффективными по сравнению с другими процессорами.
- **GPU** (графический процессор) - процессоры, специализированные для выполнения графических задач. Они обладают высокой параллельной обработкой и это где-то может быть круто, где-то наоборот.
- **DSP** (цифровой сигнальный процессор) - процессоры, специализированные для обработки сигналов в реальном времени. Они обладают высокой производительностью в области обработки сигналов.



Giga Operations Per Second (GOPS) - это единица измерения, которая используется для оценки скорости выполнения операций в вычислительной технике. Она равна одному миллиарду операций в секунду (1 000 000 000 операций в секунду)

▼ 11) Архитектура фон Неймана. Принципы. Свойства. Особенности и ограничения. Применение на практике. Машинное слово. Понятие системы команд и её роль в построении процессов. Control Unit и DataPath

Машина фон Неймана это развитие машины Тьюринга, только вместо ленты в ней используется Random Access Memory (RAM)

Особенности

1. Использование двоичного кодирования
2. Программное управление. Все команды выполняются последовательно.
 - а. Последовательность и порядок сегодня условны. (современные процессора выполняют инструкции то как они сами посчитают нужным + они еще выполняют инструкции впараллель чтоб работало быстрее)
3. Память компьютера однородно хранит данные и программы.
 - удобно, когда нет необходимости разводить по разным частям компьютера код для исполнения (программы) и сами данные которые обрабатываются.
 - не нужно делать доп. каналы данных
 - сейчас это спорно (гарвардская и принстонская архитектуры)
4. Ячейки памяти компьютера имеют адреса. RAM.
 - а. магнитная лента занимает долгое время чтоб обратиться к каким-то данным, тк большую часть времени будет уходить на то чтоб ее мотать.

- b. спорно - поскольку современные компьютеры не работают с физической памятью напрямую (а с виртуализацией)

5. Возможность условного перехода.

Виды инструкции:

1. Работа с памятью (прочитать, записать, скопировать из одного места памяти в другое)
2. арифметические , логические и битовые операции
3. управляющие операции (определить какая операция будет исполнена следующей: безусловный, условный или косвенный переход, вызовы и возврат из подпрограмм, отчасти обработка прерываний)
4. инструкции сопроцессоров: часто процессор это не единая аппаратная структура, а имеет некоторые подключаемые модули. в зависимости от того , какие платы мы подключили к процессору, у нас появляется спектр команд. особенность сопроцессоров: они имеют интерфейс взаимодействия (в отличии от встроенных инструкций) и они не имеют строгого ограничения по времени выполнения инструкций.

Применение на практике

1. В чистом виде практически не встречаются. Потому что это слишком тривиальная история, простые процессора никому не нужны. Она даже не обладает системой прирывания. Он не может эффективно реализовать ввод вывод
2. Влияние на индукстрию - трудно переоценить.
 - Если вы сегодня посмотрите на процессора, почти все из них так или иначе, либо являются фон нейманским процессорами, с некоторыми изменениями корректировками, но по сути это они либо содержат их как свои части, потому что вот эта базовый механизм управления процессора, который был сформулирован фон нейманом, оказался, во-первых очень простым. Во-вторых, он оказался очень эффективным в том плане, что его легко сделать. Он хорошо работает, и он достаточно универсален.

3. Элементы архитектуры присутствуют в подавляющем большинстве процессоров и средств разработки. И на самом деле точно примерно по такому же принципу у вас работает большинство виртуальных машин и тд.
4. Структурное программирование и ООП - наследие фон Неймана.
5. Ловушка обратной совместимости. Мы не имеем много инструментов чтоб уйти от фон Неймановской схемы. Разработчики и оборудование привыкло работать на этой модели.

Машинное слово

машинное слово - это набор символов, хранящийся в оперативном запоминающем устройстве и воспринимаемый при обработке устройством машины, как единая группа. Машинное слово служит единицей информации.

Понятие системы команд

система команд

абстрактная модель процессора, формирующая интерфейс взаимодействия между программным обеспечением и процессором

1. типы данных
2. модель памяти, система и методы адресации
3. набор инструкций
4. механизмы обработки прерываний и исключений
5. методы ввода и вывода

разнообразие систем команд

1. действие над памятью/регистрами (фон Нейман)
 - CISC произвольная сложность и длина
 - RISC фиксированный формат
2. действие без указания обрабатываемых данных (стековый)

3. управляющие сигналы процессора под видом команды (NISC)
4. группа команд под видом одной команды (VLIW)
5. действие над шиной данных (TTA)
6. команды без последовательного исполнения (редукционные)
7. система команд из одной команды (URISC)
8. управляющие сигналы процессоров под видом команды (NITTA)

ControlUnit и DataPath

Схема процессора фон Неймана практически повторяет схему информационного процессора, за исключением того, что Central Processing Unit разбит на две части: **Control Unit** и **DataPath**.

Control Unit (управляющая единица) - это компонент микроархитектуры процессора, который отвечает за управление и координацию работы процессора. Она контролирует выполнение инструкций, управляет работой DataPath и другими компонентами процессора, и обеспечивает правильную последовательность операций. Управляющая единица получает команды из памяти и декодирует их, чтобы определить, какие операции нужно выполнить.

DataPath - это ключевой компонент микроархитектуры процессора, который отвечает за выполнение операций над данными. Он содержит регистры, арифметическо-логическое устройство (АЛУ) и блоки памяти для доступа к данным. DataPath обрабатывает инструкции, вычисляет результаты и сохраняет их в нужных регистрах или памяти.

▼ **12) Гарвардская архитектура и её отличия от архитектуры фон Неймана. Принципы. Свойства. Особенности и ограничения. Применение на практике. Подходы к обходу ограничений архитектуры. Машинное слово. ControlUnit and DataPath.**

Для фон Неймановской архитектуры верно следующее:

1. Считываем команду
2. Считываем данные, которые нужны

3. Посчитали - выполнили команду

4. Вернули изменённые значения обратно в память

В чистой архитектуре фон Неймана процессор в каждый момент времени может либо читать инструкцию, либо читать/записывать единицу данных из/в памяти. В компьютере с использованием гарвардской архитектуры процессор может считывать очередную команду и оперировать памятью данных одновременно и без использования кэш-памяти

когда и инструкции и данные хранятся в одном месте, получается узкое горлышко при выполнении команд. Поэтому было бы славно, чтобы можно было читать инструкции и данные параллельно

Отсюда значит и вытекла Гарвардская архитектура, где хранилище данных и хранилище инструкций - разные физические устройства и каналы тоже физически разделены.

плюсы:

- каналы работают параллельно, не нужно блокировать какой-то при чтении другого
- могут быть разные ширины для машинного слова и адреса для данных и программы
- безопасность - изоляция памяти инструкций (не имеем доступа на запись в Program Memory)

минусы:

- сложность и стоимость выше (больше оборудования)
- перенос данных из одного хранилища в другое - сложный
- изоляция инструкций и данных (казалось бы плюс) но пример значит: делаем ссылку на функцию и всё из Data ссылаемся на Program. - безопасность испорчена

что придумали, чтобы обойти эти недостатки (обходы):

- 1) Архитектура "память инструкций, как данные" - можно читать и писать инструкции в память данных, что позволит генерировать и запускать код
- 2) Архитектура "память данных как инструкции" - реализует возможность запуска инструкций из памяти данных. Лишаемся параллельного чтения...
- 3) Модифицированная Гарвардская архитектура - доступ к памяти идет через кэши, тем самым процессор думает что доступ реализован независимо, разными каналами, но по факту все лежит в одной памяти.

машинное слово

машинное слово - это набор символов, хранящийся в оперативном запоминающем устройстве и воспринимаемый при обработке устройством машины, как единая группа. Машинное слово служит единицей информации.

ControlUnit и DataPath

Control Unit (управляющая единица) - это компонент микроархитектуры процессора, который отвечает за управление и координацию работы процессора. Она контролирует выполнение инструкций, управляет работой DataPath и другими компонентами процессора, и обеспечивает правильную последовательность операций. Управляющая единица получает команды из памяти и декодирует их, чтобы определить, какие операции нужно выполнить.

DataPath - это ключевой компонент микроархитектуры процессора, который отвечает за выполнение операций над данными. Он содержит регистры, арифметическо-логическое устройство (АЛУ) и блоки памяти для доступа к данным. DataPath обрабатывает инструкции, вычисляет результаты и сохраняет их в нужных регистрах или памяти.

▼ 13) Механизм микроопераций, микропрограммирование и его роль в развитии компьютерных систем. Особенности и ограничения.

Применение на практике. Пример оптимизации через микрокод. NISC архитектура

микрооперации - элементарные операции, из которых состоят более сложные инструкции процессора. Они представляют собой базовые действия на уровне аппаратного обеспечения, такие как чтение/запись операндов из/в регистр, операции с АЛУ

пример: Инструкция сложения на уровне машинного кода может включать в себя микрооперации чтения операндов из регистров, выполнения сложения в ALU и записи результата обратно в регистр.

микропрограммирование - метод управления выполнения инструкций в процессоре с помощью микрокода. Микрокод - низкоуровневая инструкция / набор инструкций которые управляют микрооперациями. Он определяет какие микрооперации должны быть выполнены для реализации каждой машинной инструкции

подходы к реализации Control Unit

первый - **hardwired**

у нас есть куча аппаратной логики, которая получает на вход операционный код, интерпретирует его, включает счетчик того, сколько нужно тактов на выполнение операции и отправляет управляющие сигналы

- + в том что работает достаточно просто, сводится к конечной системе управляющих автоматов, которая легко реализуется в аппаратуре.
- - в том случае если у нас большая система команд, этот автомат довольно быстро растет по сложности и будет ломаться (100% будут ошибки). а с готовым железом уже ничего не сделать кроме как выкинуть.

второй - **microwired**

в памяти есть инструкции, они проходят через некий декодер и на выходе получаем набор микроинструкций

Микроинструкция - набор управляющих сигналов к схематехнике. Вместо того, чтоб на уровне схемологии выдават сигнал (защелкнуть вентиль

регистра) . вот эти управляющие сигналы уже описаны в рамках программы (то есть мы можем их переписать а не выбрасывать схему в случае ошибки)

микропрограммное управление

плюсы:

- простота реализации cisc (реализовывать более сложные инструкции, что упрощало программирование на уровне машинного кода)
- программный код писать легче, чем делать комбинационные схемы
- у программиста есть доступ к микрокоду
- генерации системы команд под задачу
- Микрокод обеспечивал совместимость между различными поколениями процессоров. Новые процессоры могли эмулировать поведение старых через микрокод, облегчая переход на новые технологии.

минусы:

- хранение микрокода в процессоре - занимает место (все микрокоманды должны храниться непосредственно в процессоре чтоб доступ к ним был за один такт, + микрокод будет очень разряженным, тк будет много бесполезных нулей чисто)
- программисту нужно много учиться запоминать большую систему команд
- Разнообразие архитектур → теперь компилятор нужен не на один процессор а на 10, соответственно они станут хуже
- Разнообразие команд усложняет оптимизацию процессора (много способов реализовать одно и тоже поведение кучей разных микрокоманд)
- Микрокод приносит все проблемы программирования (объем микрокода невероятно огромный, а разрабатывать новые невероятно сложно так как система команд процессора большая)

пример оптимизации через микрокод:

может служить процессор CISC (Complex Instruction Set Computer), который использует микрокод для эмуляции инструкций RISC (Reduced Instruction Set Computer) на аппаратном уровне. В этом случае, сложные инструкции CISC разбиваются на более простые инструкции RISC, которые затем выполняются аппаратно. Это позволяет улучшить производительность и эффективность процессора, поскольку каждая инструкция RISC может быть выполнена быстрее и эффективнее, чем соответствующая инструкция CISC

архитектура NISC (no instruction set computing)

Появилась как упрощение CISC, суть которой состоит в том что мы полностью отказываемся от инструкций и оставляем только микрокод. Так как каждая инструкция состоит из n микрокоманд, то в NISC размер программы увеличивается на порядки

В памяти теперь напрямую хранится код для управления вентилями

Достоинства:

1. Упрощение аппаратуры - полностью избавляемся от слоя памяти инструкций (по сути убрали интерпретатор инструкций)
2. Максимальная эффективность программного управления (компиляторы умные, они сами используют нашу аппаратуру эффективно)
3. Нет набора команд, нет проблем с тем что их надо проектировать

Недостатки:

1. Нет бинарной совместимости, то есть при изменении структуры процессора, все существующие программы не будут работать
2. Низкая плотность машинного кода - так как у нас каждая команда состоит из битов для управления вентилями, нам постоянно приходится хранить ненужные 0 (как это фиксируется: что в ЦИСК что в НИСК-е к процессору подключается программный архиватор, который

получает архивированные инструкции и налету их распаковывает в управляющие сигналы. иначе объем данных становится чудовищным. но при этом усложняется логика)

Использование:

- Применяется в ускорителях, в высокоуровневом синтезе (HLS), спец. вычислителях

▼ 14) что такое CISC? Роль в развитии компьютерных систем. Применение на практике. Достоинства и недостатки. Отличия от архитектуры фон Неймана. Особенности программирования

CISC - тип архитектуры процессора, при котором одна инструкция = несколько низкоуровневых команд (так называемых микроопераций) таких как загрузка из памяти, арифметические операции, сохранение в память или выполнять сложные операции в несколько шагов за один такт

Причины появления:

- Низкоуровневые языки: раньше писали на ассемблере и было много кода
- Разнообразие архитектур: оч много разных процессорных архитектур
- Незрелость компиляторов. так удобнее стало программировать
- Высокая производительность стала.

Недостатки

- Сложная система команд (использование, анализ). (чем сложнее инструментарий для программиста, тем хуже качество кода)
- Сложное устройство процессора и Control Unit. (чем сложнее система команд, тем сложнее контроллеру читать команды, например момент с тем, откуда брать значение (либо это просто число либо адрес))
- Сложно генерировать эффективный машинный код.

Достоинства:

1. **Богатый Набор Инструкций:** Позволяет писать более компактный и высокоуровневый машинный код.
2. **Упрощение Программирования:** Более сложные инструкции облегчают написание эффективного кода на языке ассемблера.
3. **Уменьшение Нагрузки на Компилятор:** За счет богатства инструкций, компилятору проще оптимизировать код.

отличия фон Неймана от CISC

честно говоря вопрос такой не оч понятный, потому что это как сравнивать цвет и форму. что я имею в виду:

Архитектура фон Неймана — это концепция компьютерной архитектуры, в которой программы и данные хранятся в одной и той же памяти и обрабатываются одним и тем же набором инструкций. Она описывает общую структуру компьютера, но не детали реализации инструкций.

CISC, с другой стороны, относится конкретно к проектированию набора инструкций процессора. Тут это как раз про Control Unit. Таким образом, компьютер, основанный на архитектуре фон Неймана, может использовать как CISC, так и RISC (Reduced Instruction Set Computing), так и HISC) подходы к проектированию своего набора инструкций.

особенности программирования

программирование для CISC-архитектур - использование большого набора инструкций, который может быть особенно полезен в низкоуровневом программировании. Однако большинство современных приложений пишутся на высокоуровневых языках программирования, где различия между CISC и RISC становятся менее заметными, так как компиляторы автоматически оптимизируют код для конкретной архитектуры

▼ 15) что такое RISC? Роль в развитии компьютерных систем. Применение на практике. Достоинства и недостатки. Отличия от архитектуры фон

Неймана. Особенности программирования.

RISC (reduced instruction set computer)

Основная идея RISC заключается в том, что большинство инструкций выполняются за один цикл процессора, что позволяет увеличить производительность процессора.

Для достижения этого, RISC-процессоры обычно имеют большое количество регистров и используют технику, называемую конвейеризацией, для выполнения нескольких инструкций одновременно. Тут так кодируются инструкции, чтобы их декодирование было более простым, а время выполнения - меньшим.

Ключевой технологией для RISC процессоров стали *языки высокого уровня*, которые позволили разработчикам ПО уйти от написания исходного кода на уровне ассемблера, а значит вопросы удобства написания кода сменились вопросам удобства генерации кода.

Инструкции CISC могут быть любой длины. Максимальная теоретическая длина инструкции x86 может быть бесконечной, но на практике не превышает 15 байт. **Инструкции RISC имеют ограниченную длину.**

Конвейеризация в RISC процессорах позволяет одновременно выполнять несколько инструкций, разбивая их на отдельные этапы и обрабатывая их параллельно. Это увеличивает производительность процессора и позволяет более эффективно использовать его ресурсы. Однако, конвейеризация также может привести к появлению задержек и конфликтов, которые могут снизить производительность в некоторых ситуациях.

Лучше сделать минимальный набор унифицированных команд ориентированных на быстрое исполнение, а "сложное поведение" реализовывать их последовательностями. Отсюда:

Достоинства:

- освободить в процессоре площадь от элементов сложных декодеров команд, что в частности позволило расширить регистровые файлы;

- освободить большое количество памяти микрокоманд для кеша команд, что отчасти компенсировало необходимость более частой выборки команд;
- добиться малого количества быстро исполняемых команд;
- "однообразность" команд открыла возможность к оптимизациям производительности на уровне инструкций

Недостатки:

- **Большее число инструкций:** В архитектуре RISC для выполнения некоторых операций может потребоваться больше инструкций, чем в архитектуре CISC (Complex Instruction Set Computer). Это может привести к увеличению размера и сложности программного кода 2.
- **Высокая загрузка памяти:** Поскольку каждая инструкция в архитектуре RISC выполняется за один цикл процессора, это может привести к увеличению трафика памяти, поскольку процессор должен постоянно извлекать новые инструкции из памяти 2.
- **Ограниченное количество регистров:** Хотя большое количество регистров в архитектуре RISC может улучшить производительность, оно также может усложнить схему декодирования номера регистра, тем самым увеличивая время доступа к регистрам 2.
- **Ограниченная универсальность:** Некоторые сложные операции могут быть более сложными для реализации в архитектуре RISC, поскольку они могут потребовать нескольких простых инструкций, вместо одной сложной инструкции, как в архитектуре CISC

отличия фон Неймана от RISC

честно говоря вопрос такой не оч понятный, потому что это как сравнивать цвет и форму. что я имею в виду:

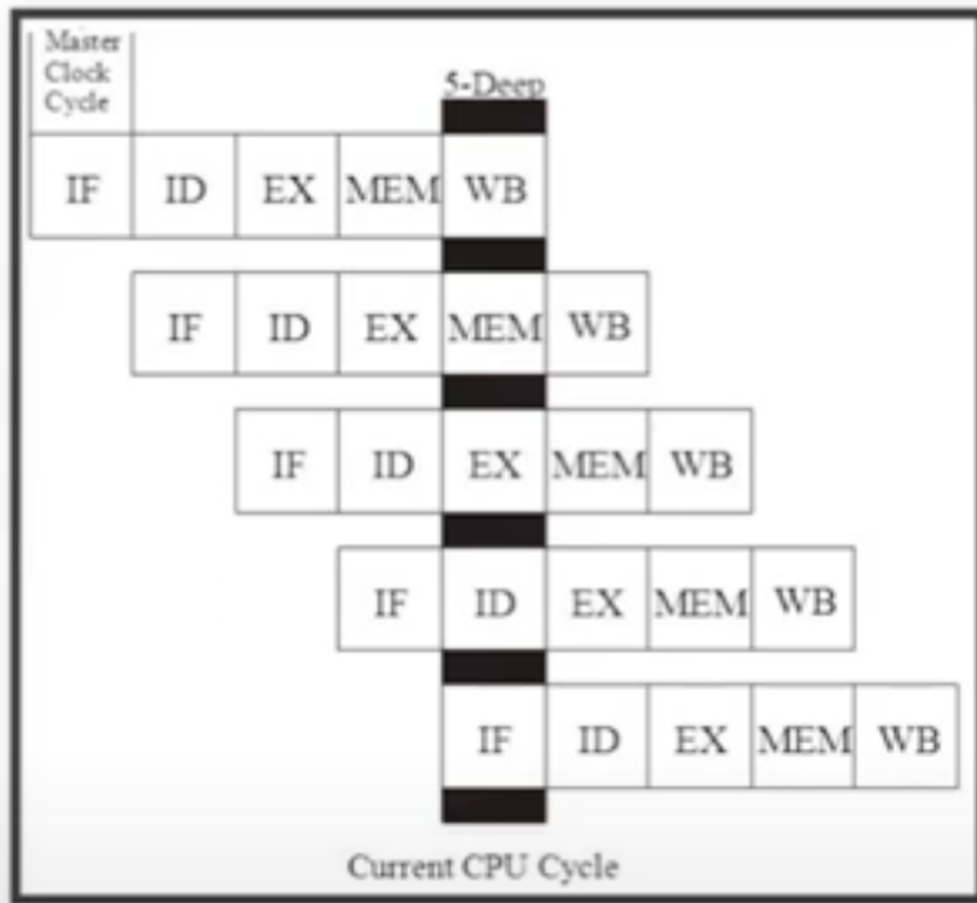
Архитектура фон Неймана — это концепция компьютерной архитектуры, в которой программы и данные хранятся в одной и той же памяти и обрабатываются одним и тем же набором инструкций. Она описывает общую структуру компьютера, но не детали реализации инструкций.

RISC, с другой стороны, относится конкретно к проектированию набора инструкций процессора. Тут это как раз про Control Unit. Таким образом, компьютер, основанный на архитектуре фон Неймана, может использовать как CISC, так и RISC (Reduced Instruction Set Computing), так и HISC) подходы к проектированию своего набора инструкций.

▼ 16-17) Конвейеризированное исполнение команд. Стадии конвейера. Виды конфликтов (по данным, по управлению), их примеры и влияние на производительность. Достоинства и недостатки.

Шаги конвейеризации:

1. Извлечение. Инструкция извлекается из памяти
2. Декодирование. Инструкция декодируется, то есть определяется, что эта инструкция делает
3. Выполнение. Инструкция выполняется, используя арифметико-логическое устройство или совершая сдвиговые операции
4. Доступ к памяти. Если необходимо происходит доступ к памяти (ещё за один операндом топ-топ)
5. Запись результатов. Результаты записываются в соответствующий регистр



Эти шаги могут выполняться одновременно для разных инструкций, что позволяет процессору выполнять несколько инструкций за один цикл. Это увеличивает производительность процессора, поскольку он может начать выполнение следующей инструкции, прежде чем предыдущая инструкция будет завершена

Конфликты:

1. структурные конфликты - мы не можем выполнить некоторые сочетания инструкций.
 - а. пузырьёк - вместо одной из команд взаимодействия с памятью пишем нолик. Тогда ничего не сработает на эту команду. Но в

некоторых случаях может быть оооочень много 0 и процессор часто будет простаивать

b. Гарвардская архитектура

2. Конфликты по данным

a. RAW - (read after write) там ещё всякие комбинации RAR, WAW, WAR - проблема, возникающая при последовательном использовании одного и того же регистра в двух командах. 1-ая команда: что-то посчитать и записать в регистр, 2-ая команда: использовать этот же регистр, например, для сложения. При конвейерной обработке значение этого регистра для сложения возьмется старое, а не новое. Решения:

- i. исполнение не по порядку - разносим команды в разные места в памяти (делают это компиляторы и процессоры)
- ii. в процессе компиляции можно переименовывать регистры, это устранил проблему (WAW)
- iii. вставка пузыря
- iv. Пропуск операндов - вместо того, чтобы сохранять промежуточные данные в регистры для использования их другими командами, мы можем на лету выхватывать их

3. Конфликты по управлению - конфликты возникают при появлении условных и безусловных переходов. При выполнении команды мы увидели, что нам надо прыгнуть на 100 команд вперед, а в конвейер уже загружены следующие 5 команд - проблема. Решение:

- a. Пузырьком, как только видимо в IF, что у нас инструкция перехода, начинаем заполнять конвейер дальше пузырями. Плохо - из-за простаивания
- b. Сброс конвейера - работает, но много сложной логики

Достоинства конвейера:

- повышение производительности и уровня утилизации ресурсов.

- Эффективное использование ресурсов процессора: Разные стадии инструкций обрабатываются разными компонентами процессора одновременно.

Недостатки:

- Конфликты и задержки: Как было упомянуто выше, конфликты могут привести к задержкам, что уменьшает эффективность конвейера.
- Сложность дизайна: Конвейеризация усложняет архитектуру процессора и алгоритмы управления конвейером.
- Непредсказуемое время исполнения
- Не все операции за один машинный цикл

▼ 18) Что такое SOP (Stack Oriented Processor, стековый процессор)? Роль в развитии компьютерных систем. Применение на практике. Достоинства и недостатки. Отличия от архитектуры фон Неймана. Особенности программирования

SOP или стековый процессор - тип микропроцессора, в котором обработка происходит не в регистрах, а на стеке (может быть нескольких).

микропроцессор - процессор, реализованный в виде одной микросхемы или комплекта из нескольких специализированных микросхем. Это устройство, отвечающее за выполнение арифметических, логических операций и операций управления, записанных в машинном коде

Те у нас отдельно есть память, есть АЛУ и есть стек. Соответственно есть только операции push и pop

Исторический Контекст: Стековые процессоры были популярны в 1970-х и 1980-х годах, особенно в академических кругах и для обучения принципам программирования. Такой тип процессоров оказал широкое влияние на микроэлектронику в СССР. Стековые процессоры имеют гораздо меньшую требовательность к памяти и низкое энергопотребление, по сравнению, например с RISC процессором.

Эти языки программирования использующие SOP обычно имеют постфиксную или обратную польскую нотацию, где любые аргументы или

параметры для команды указываются перед этой командой

На практике, использование SoP может быть полезно в ситуациях, когда требуется быстрое выполнение простых операций и минимизация использования памяти. Однако, это может быть менее эффективным, чем использование процессоров, которые используют регистры для хранения и обработки данных

Достоинства

1. **Простота реализации:** Меньше требуется аппаратных ресурсов по сравнению с регистровыми процессорами.
2. **Компактность кода**
3. **Простота компиляции:** Упрощает компиляцию некоторых языков программирования.

Недостатки

1. Низкая эффективность **при большом количестве данных** на стеке
2. **Невозможность использовать параллелизм уровня инструкций:**
Аппаратно реализованный стек подразумевает строгую последовательность происходящих событий
3. **Сложность программирования:** Неудобство в программировании, особенно для сложных алгоритмов
4. **Меньшая популярность:** Меньшая поддержка и разработка по сравнению с более традиционными архитектурами.

Отличия от архитектуры фон Неймана

- **Модель Данных:** В фон Неймановских архитектурах операции выполняются над данными, чаще хранящимися в регистрах или памяти но там тоже может быть стек, в то время как в стековых процессорах – над данными на вершине стека.
- **Программирование и Компиляция:** В стековых процессорах программирование связано с работой со стеком, в то время как фон Неймановские системы более гибки в этом отношении.

Особенности программирования

- **Языки Программирования:** используются специализированные языки, оптимизированные для стековых операций, например, Forth.
- **Стиль Кодирования:** Программирование требует учета стековой природы выполнения команд, что отличается от традиционного подхода программирования.

▼ 19) Поддержка операций ввода/вывода в фон Неймановских процессорах. Поддержка на уровне системы команд (порт, отображение в память). Программно-управляемый ввод-вывод. Ввод-вывод через механизм прерываний. Механизм прямого доступа к памяти.

В данном контексте ввод-вывод это передача информации между процессором (CPU) и внешними устройствами (диски, мышки, экраны и т.п.).

Есть следующие типы поддержки операций ввода/вывода:

1. ввод/вывод через память
2. ввод/вывод через порты

поподробнее про каждый:

1) При **вводе-выводе через память** часть адресного пространства резервируется для устройств ввода вывода, поэтому адресное пространство используется как для доступа к основной памяти, так и для доступа к устройствам ввода-вывода.

Ключевые **достоинства** данного подхода:

- Упрощение внутреннего устройства процессора.
- можем работать с данными устройств ввода-вывода как с обычными данными
- Удобнее работа с функциями, которые оперируют большими объёмами данных (например, чтение-запись на диск, в видеоадаптер). Нет необходимости "двойного переноса".

К **недостаткам** можно отнести:

- использование единой шины для ввода-вывода и доступа к памяти;

- неоднородность памяти, сложная конфигурация системы (включая инструментарий);
- конфликты с системой кеширования и параллелизмом уровня инструкций;
- запись данных в кеш не гарантирует, что они будут отправлены в устройство ввода-вывода;
- устройства ввода-вывода должны работать с полным адресом (если 64 бита, то всеми 64 битами), чего не требуется с точки зрения ввода-вывода;
- возможно замедление работы системы в целом, так как ввод-вывод зачастую медленнее чем доступ к памяти.

2) При **вводе-выводе через порты** адресация устройств ввода-вывода производится независимо от адресации памяти, но для этого требуется наличие специально выделенных для этого инструкций.

Достоинства:

- Объем логики управления соответствует адресному пространству устройств ввода/вывода.
- На уровне ассемблера чётко видно, где реализуется доступ к памяти, а где к внешним устройствам.
- Адресное пространство однородно и не имеет "запретных мест".
- В целом система проще, так как ввод-вывод не приходится интегрировать в систему кешей, а операции ввода/вывода могут обрабатываться особым образом на уровне конвейеров и спекулятивных вычислений.

Недостатки:

- Усложнение системы команд и процессора, реализующую его.
- Данные ввода-вывода становятся данными второго сорта. Когда у нас есть отдельная память для IO, то любое устройство ввода-вывода

запишет инфу сначала туда, а потом, чтобы с этими данными поработать нам надо будет их загрузить в основную память

сами ввод/вывод

1. **Программно-управляемый ввод-вывод** – все операции реализуются процессором, включая постоянное наблюдение за готовностью внешнего устройства к передаче данных. Требует огромных временных затрат на взаимодействие с внешним устройством. Процессор в цикле (polling) опрашивает состояние внешнего устройства и по наступлению необходимости начинает с ним взаимодействовать.
2. **ввод/вывод через механизм прерываний** - Снимает с процессора задачу постоянного наблюдения за внешним устройством и позволяет это реализовать по внешнему событию, к примеру готовность устройства к передаче данных или наступление заданного события (срабатывание таймера, нажатие на клавишу и т.п.). Непосредственно ввод-вывод реализуется по прежнему процессором.

Получив прерывание, процессор незамедлительно прерывает текущий поток управления и передает управление по вектору прерывания, сохраняя состояние прерванного потока, а после завершения, возвращается к нему, восстанавливая состояние исходного.

В зависимости от **источника возникновения сигнала** прерывания делятся на:

- **асинхронные**, или внешние (аппаратные) – события от ВУ, могут произойти в произвольный момент относительно внутренней работы процессора
- **синхронные**, или внутренние – события в самом процессоре как результат нарушения каких-то условий при исполнении машинного кода, например, деление на ноль или переполнение стека.

По **принятию решения о необходимости обработки прерывания** прерывания делятся на:

- **Маскируемые прерывания** (разработчику доступны регистры, конфигурация которых позволяет их игнорировать). К примеру: отключение прерываний в рамках выполнения критической секции кода.
- **Немаскируемые прерывания** (которые нельзя проигнорировать). К примеру: ошибка доступа к основной памяти или обработки сторожевого таймера

Сторожевой таймер — *аппаратно* реализованная схема контроля над зависанием системы. Он используется для обнаружения компьютерных неисправностей и восстановления после них.

3. механизм прямого доступа к памяти - часто появляется необходимость просто считать данные с у-ва ввода-вывода в память системы. Если это реализовывать через систему прерываний, то на время записи данных с ВУ в основную память наша система будет простаивать (ВУ → Проц → Память). Для того, чтобы освободить процессор от такой необходимости был придуман механизм прямого доступа к памяти (DMA - direct memory access). Он предлагает поставить между CPU и ВУ контроллер, который будет выполнять команды переноса данных между памятью ВУ и основной памятью. В интерфейсе этого контроллера есть: количество слов, адрес в памяти для записи, control register (режим передачи данных). Так как в конечном итоге все данные на основную память попадают через одну шину, то необходимо определиться с принципом параллельного взаимодействия и CPU и контроллера DMA с этой самой шиной.

Выделяют **два основных принципа работы**:

- **Third-party**, где работа с DMA управляется полностью процессором, и любая передача данных должна инициализироваться им.
- **Bus mastering**, где работа с DMA управляется в том числе и со стороны устройств ввода-вывода, что позволяет инициализировать передачу данных без участия процессора.

Так как в подавляющем большинстве случаев одновременная работа контроллера прямого доступа с памятью и процессора невозможна, **режимы функционирования DMA по отношению к памяти**:

- **Пакетный режим (Burst Mode).** Передача данных осуществляется единой операцией, которая не может быть прервана процессором.
- **Циклический режим (Cycle stealing mode).** Работа с памятью для процессора и устройств ввода-вывода имеет циклическую природу, где для каждого вида операций выделяется свой временной слот. Это позволяет избежать непредсказуемых задержек в работе процессора, но может снижать общую производительность системы.
- **Прозрачный режим (Transparent Mode).** Передача данных осуществляется в те моменты времени, когда процессор не взаимодействует с памятью.

▼ **20) Параллелизм уровня задач. Кооперативная многозадачность. Принцип работы и подходы к реализации. Примеры использования. Достоинства и ограничения. Зелёные процессы. Проблема синхронизации процессов по управлению и по данным. Сравнение с альтернативами.**

Параллелизм уровня задач - выполнения различных задач одновременно, каждая из которых может быть выполнена независимо от других. Этот тип параллелизма часто используется в сценариях, где есть множество независимых задач, которые могут быть выполнены параллельно для увеличения производительности и эффективности системы. Один из способов обеспечить такой параллелизм - кооперативная многозадачность (второй - вытесняющий подход, это через прерывания реализовано)

Кооперативная многозадачность - Тип многозадачности, при котором следующая задача выполняется только после того, как текущая *задача явно объявит себя готовой отдать процессорное время* другим задачам. Такой подход называется "кооперативным" потому, что он требует явного взаимодействия между задачами и операционной системой

По факту Кооперативная многозадачность - это тип многозадачности, при котором задачи выполняются параллельно, но не конкурируют друг с другом за ресурсы процессора. Вместо этого, задачи добровольно

уступают процессор друг другу. На уровне процесса задачи дробятся на мелкие части и запускаются как отдельные процессы.

Способы реализации:

- **В рамках операционной системы:** предоставляется набор специализированных системных вызовов, которые позволяют сообщить ОС о том, что поток команд может быть приостановлен.
- **На уровне виртуальных машин и/или компилятора.** Такие конструкции языков программирования как *yield*, *async/await* и т.п.
- **В ручном режиме,** когда многозадачность реализуется программистом самостоятельно в рамках его приложения

Примеры использования

В современном мире на уровне ОС такая многозадачность не встречается. Реализацию такой многозадачности можно увидеть в современных ЯП высокого уровня (GoLang, Erlang). Они реализовывают легковесную многопоточность именно с помощью кооперативной многозадачности. Кроме того это можно увидеть в рантаймах многих интерпретируемых языков (интерпретатор выполняет команду за командой поэтому может остановиться в любой момент). Ну и всякие *async-await* и *callback*-и (корутины))) тоже являются примерами кооперативной многозадачности.

На практике кооперативная многозадачность встречается в:

- 1) Ситуациях когда надо освободить ЦПУ на время IO
- 2) Простых встроенных системах (неудобно делать любую другую многозадачность, т.к. это самая легкая, а контроллер у нас маленький и простенький)
- 3) Статическом планировании (буквально до запуска системы распланировали порядок выполнения)
- 4) Оптимизация систем, в которых задачи необходимо часто переключать

Преимущества:

- нет необходимости защищать все разделяемые структуры данных объектами типа критических секций и мьютексов (*Основная цель мьютекса — обеспечить, чтобы только один поток (или процесс) мог иметь доступ к критическому ресурсу или критической секции*), что упрощает программирование и скорость вычислений (примечание: сохраняются классические сложности работы с изменяемым состоянием);

Недостатки:

- Небезопасность и непредсказуемость отдельных потоков команд. В случае если процесс не отдаёт управление в течение долгого времени это может быть обусловлено как "особенностями реализации", так и "ошибкой программирования". В любом случае это вносит непредсказуемость в поведение компьютера в целом.

Зелёные потоки - процессы, управление которыми отдано не ОС, а виртуальной машине. Управляются в пространстве пользователя, а не в пространстве ядра. Это позволяет им работать там, где нет встроенной поддержки потоков. Они эмулируют многопоточную среду не обращаясь к реализации легковесных потоков самой ОС. Да, под капотом это все - **один поток ОС**, они используют его все вместе посредством параллелизма. И здесь как раз так работает кооперативная многозадачность, которая распределяет все эти процессы между собой и быстренько их переключает.

Когда зеленый поток использует блокирующий системный вызов, блокируются все потоки внутри процесса. Поэтому они должны использовать асинхронные операции ввода/вывода.

Вытесняющая многозадачность

1. Управление:

- Управление процессами осуществляется операционной системой.

Кооперативная многозадачность

1. Управление:

- Задачи самостоятельно контролируют передачу управления, что требует

- Переключение задач происходит автоматически и независимо от действий программ.

2. Отзывчивость:

- Высокая, так как ОС может прервать процесс в любой момент для переключения на другой процесс.

3. Ресурсоемкость:

- Более ресурсоемкая из-за необходимости частых контекстных переключений.

4. Сложность разработки:

- Более сложная для разработчиков, так как требует учета синхронизации и конкуренции за ресурсы

включения механизмов переключения задач в код приложения.

2. Отзывчивость:

- Может быть ниже, так как один процесс может удерживать управление на себе, не позволяя другим процессам выполняться.

3. Ресурсоемкость:

- Менее ресурсоемкая, так как происходит меньше контекстных переключений.

4. Сложность разработки:

- Может быть проще в реализации для отдельных приложений, но требует большей ответственности от разработчиков за управление временем CPU.

▼ 21) Параллелизм уровня задач. Вытесняющая многозадачность.

Механизмы переключения задач. Примеры использования. Достоинства и ограничения. Проблема синхронизации процессов по управлению и по данным. Сравнение с альтернативами.

Параллелизм уровня задач - выполнения различных задач одновременно, каждая из которых может быть выполнена независимо от других. Этот тип параллелизма часто используется в сценариях, где есть множество

независимых задач, которые могут быть выполнены параллельно для увеличения производительности и эффективности системы.

Вытесняющая многозадачность - механизм многозадачности при котором процессы переключаются по прерываниям или же "внешним событиям". При такой многозадачности у нас всегда есть **планировщик**, который менеджит процессорное время. Механизм остановки задачи меняется на **механизм прерывания процесса**, который прерывает задачу в не зависимости от её желания. Важно отметить, что в таком случае исполняемая программа может прерваться буквально в любой момент.

Этот вид многозадачности обеспечивает *более быстрый отклик на действия пользователя.*

Система прерываний позволяет по событию переключиться к на заданную инструкцию и вернуться назад по окончании обработки. Источники прерываний бывают двух видов: **аппаратные**, которые делятся на **внешние(нажатие клавиши, сетевой пакет) и внутренние(деление на 0, ошибка доступа памяти)**, **программные** - взаимодействие программы и ОС.

Примеры использования

- **Операционные системы:** Практически все современные операционные системы (например, Windows, macOS, Linux) используют вытесняющую многозадачность для управления приложениями и системными процессами
- **Серверные системы:** В серверных средах вытесняющая многозадачность позволяет обрабатывать множество параллельных запросов, например, веб-запросов или транзакций баз данных
- **Мобильные Устройства:** Мобильные операционные системы, такие как Android и iOS, также используют вытесняющую многозадачность для управления приложениями и системными службами

Преимущества

- простота разработки ПО с одним процессом

- контроль за ресурсами со стороны ОС. Изоляция
- интерактивность системы

Недостатки

- лишние переключения: Вытесняющая многозадачность может приводить к частым и иногда ненужным переключениям контекста
- разделяемые состояния и непредсказуемые переключения (синхронизация гонки)
- не знаем сколько будет длиться в итоге программа

Синхронизация управления нужна, чтобы обеспечить правильный порядок операций. Это особенно важно в системах, где порядок выполнения может повлиять на результат работы программы.

- **Примеры Проблем:**

- **Взаимная блокировка (Deadlock):** Происходит, когда два или более процессов ждут друг друга, создавая бесконечное ожидание.
- **Голодание (Starvation):** Когда один или несколько процессов не получают доступ к ресурсам из-за конкуренции.
- **Состояние гонки (Race Condition):** Возникает, когда результат выполнения программы зависит от неопределенного порядка выполнения процессов или потоков.

- **Решения:**

- **Семафоры и мьютексы:** Используются для управления доступом к ресурсам.
- **Условные переменные:** Помогают организовать ожидание определенных условий.
- **Барьеры:** Синхронизируют группы процессов в определенных точках выполнения.

Синхронизация данных обеспечивает целостность и последовательность доступа к данным, которые используются несколькими процессами или потоками.

- **Проблемы:**

- **Нарушение целостности данных:** Происходит, когда несколько процессов одновременно изменяют одни и те же данные.
- **Несогласованное чтение:** Когда один процесс читает данные во время их изменения другим процессом.

- **Решения:**

- **Блокировки:** Предотвращают одновременный доступ к данным.

перейдём к сравнению

по сравнению с кооперативной многозадачностью у нас появляется контроль над процессами из вне, мгновенная реакция и тд, но при этом нам надо запариваться над обработкой прерываний, стараться синхронизировать общие ресурсы

Вытесняющая многозадачность

1. Управление:

- Управление процессами осуществляется операционной системой.
- Переключение задач происходит автоматически и независимо от действий программ.

2. Отзывчивость:

- Высокая, так как ОС может прервать процесс в любой момент для переключения на другой процесс.

3. Ресурсоемкость:

- Более ресурсоемкая из-за необходимости частых контекстных переключений.

4. Сложность разработки:

- Более сложная для разработчиков, так как требует учета синхронизации и конкуренции за ресурсы

Кооперативная многозадачность

1. Управление:

- Задачи самостоятельно контролируют передачу управления, что требует включения механизмов переключения задач в код приложения.

2. Отзывчивость:

- Может быть ниже, так как один процесс может удерживать управление на себе, не позволяя другим процессам выполняться.

3. Ресурсоемкость:

- Менее ресурсоемкая, так как происходит меньше контекстных переключений.

4. Сложность разработки:

- Может быть проще в реализации для отдельных приложений, но требует большей ответственности от разработчиков за управление временем CPU.

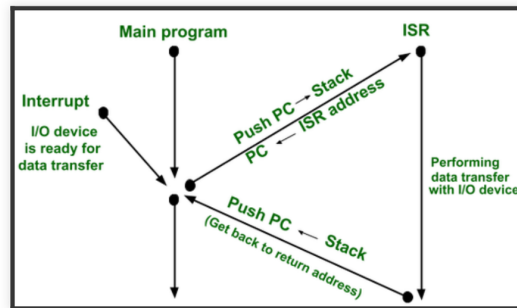
▼ 22) Система прерываний. Виды прерываний. Механизм обработки прерываний по шагам. Задачи, решаемые механизмом прерываний. Сторожевой таймер

Система прерываний позволяет переключиться (совершить переход к заданной инструкции) по событию и вернуться назад по окончании обработки.

Прерывания бывают **маскируемые** - их можно запретить установкой флага и **немаскируемые** - всегда работают. Кроме того прерывания также делятся на абсолютные - прервут любое другое (прим: сбой в памяти) и относительные - просто менеджерятся по приоритетам.

Механизм обработки прерываний по шагам

1. Выполнение основного потока.
2. Запрос прерывания.
3. Сохранение адреса возврата.
4. Вызов обработчика прерывания (ISR) — обычного кода в специальном месте.
5. Завершение ISR.
6. Восстановление адреса возврата.
7. Продолжение основного потока.



Система прерываний обеспечивает минимум два уровня задач:

1. Основной поток исполнения.
2. Поток (потoki) обработчиков прерываний.

задачи, решаемые механизмом прерывания задач

Механизм прерываний позволяет решать такие задачи как: обработка событий внешнего мира, ввод-вывод, обработка исключительных ситуаций (если мы поделим на Олик сработает прерывание и система это обработает, при кооперативной многозадачности поведение может быть непредсказуемым).

Сторожевой таймер — аппаратно реализованная схема контроля над зависанием системы. Он используется для обнаружения компьютерных неисправностей и восстановления после них.

Он имеет возможность послать сигнал прерывания для сброса (Reset) на процессор, процессор в свою очередь раз в какое-то время(мб раз в какое-то кол-во тактов) шлет сигнал этому таймеру по типу: "у меня все ок, я работаю". Если же процессор не уложится в нужное время и подобный сигнал придет позже(что явно говорит о каких-то проблемах на

процессоре), то сторожевой таймер отправит сигнал прерывания для сброса процессора.

▼ 23) Параллелизм уровня задач. Проблемы совмещения, изоляции и взаимодействия задач между собой. Методы решения данных проблем (с точки зрения программиста и пользователя): распределение по адресному пространству, банки памяти, сегментная организация памяти и виртуальная память.

Параллелизм уровня задач - выполнения различных задач одновременно, каждая из которых может быть выполнена независимо от других. Этот тип параллелизма часто используется в сценариях, где есть множество независимых задач, которые могут быть выполнены параллельно для увеличения производительности и эффективности системы.

Проблема совмещения заключается в том, что задачи могут пытаться использовать общие ресурсы, такие как процессор, память или устройства ввода-вывода.

Проблема изоляции заключается в том, что задачи должны быть изолированы друг от друга, чтобы они не могли вносить изменения в данные, принадлежащие другим задачам.

Проблема взаимодействия заключается в том, что задачи должны иметь возможность взаимодействовать друг с другом, чтобы обмениваться данными или совместно использовать ресурсы.

распределение по адресному пространству:

- Распределение по адресному пространству, заключается в том, чтобы распределить адресное пространство задач таким образом, чтобы они не могли обращаться к данным друг друга.

банки памяти:

- реализуют изоляцию с помощью двухэтапного доступа к памяти: сначала выбираем банк, потом выбираем адрес внутри этого банка.

Таким образом можем изолировать программы и данные друг от друга. По сути своей это костыль и банки используются для изоляции очень редко. Основное применение - расширение машинного слова, т.е. две банки параллельно ($8 + 8 = 16$), либо для расширения памяти (не влезает в адресное пространство), либо переключение работы (BIOS)

Сегментная память

- идея разметки памяти процессора по назначению. Режем память на сегменты с четкими адресами начала и конца, но разными размерами в зависимости от ситуации, а процесс уже всегда работает внутри этого сегмента. Таким образом все наши процессы становятся независимыми. При обращении процессора к памяти он отправляет номер сегмента и смещение относительно начала этого сегмента. С помощью таблицы сегментов мы получаем физический адрес памяти и обращаемся к нему. При выходе за границы сегмента - все падает. Таким образом нам не нужно волноваться: "а не залезет ли кто ко мне в память?", сегментация просто не даст этого сделать.

В этом случае у самих процессов появляются права доступа к сегментам, следовательно у нас появляется возможность создавать общие сегменты (буферы строго фиксированного размера), что помогает шарить данные.

В целом, сегментация сама по себе позволяет нам гарантировать строго выполнение инструкций по заданным маршрутам, все счастливы

Но у нее есть и

минусы: внешняя фрагментация при освобождении сегментов (при динамическом размере сегментов), внутренняя сегментация (при фиксированном размере сегментов). Кроме того, такая память сложная в поддержке со стороны ПО (именно поэтому сейчас все используют вирт. память)

виртуальная память

- это технология, которая позволяет компьютеру использовать больше памяти, чем физически установлено в системе. Она работает путем автоматического перемещения частей программы между основной памятью (RAM) и вторичным хранилищем, таким как жесткий диск. Виртуальная память режет все виртуальное адресное пространство на страницы и отображает используемые страницы в физическую память. А если используемых страниц больше чем физической памяти, то мы выгружаем память на диск пока она нам снова не понадобится.

Плюсы:

- Освобождает программиста от необходимости вручную управлять загрузкой частей программы в память и согласовывать использование памяти с другими программами
- Освобождает программиста от необходимости вручную управлять загрузкой частей программы в память и согласовывать использование памяти с другими программами
- В многозадачных системах изолирует выполняющиеся программы друг от друга путем назначения им непересекающихся адресных пространств

Минусы:

- Переключение между приложениями может занять больше времени
- непредсказуемая длительность доступа к памяти(физически информация может лежать на магнитной ленте, потому что мы ее туда убрали до "востребования")

▼ 24) Сегментная и виртуальная памяти. Решаемые задачи и принципы работы. Проблема фрагментации. Достоинства и недостатки.

Сегментная память

- идея разметки памяти процессора по назначению. Режем память на сегменты с четкими адресами начала и конца, но разными размерами в зависимости от ситуации, а процесс уже всегда работает внутри этого сегмента. Таким образом все наши процессы становятся независимыми. При обращении процессора к памяти он отправляет

номер сегмента и смещение относительно начала этого сегмента. С помощью таблицы сегментов мы получаем физический адрес памяти и обращаемся к нему. При выходе за границы сегмента - все падает. Таким образом нам не нужно волноваться: "а не залезет ли кто ко мне в память?", сегментация просто не даст этого сделать.

В этом случае у самих процессов появляются права доступа к сегментам, следовательно у нас появляется возможность создавать общие сегменты (буферы строго фиксированного размера), что помогает шарить данные.

В целом, сегментация сама по себе позволяет нам гарантировать строго выполнение инструкций по заданным маршрутам, все счастливы

Но у нее есть и

минусы: внешняя фрагментация при освобождении сегментов (при динамическом размере сегментов), внутренняя сегментация(при фиксированном размере сегментов). Кроме того, такая память сложная в поддержке со стороны ПО(именно поэтому щас все используют вирт. память)

виртуальная память

- это технология, которая позволяет компьютеру использовать больше памяти, чем физически установлено в системе. Она работает путем автоматического перемещения частей программы между основной памятью (RAM) и вторичным хранилищем, таким как жесткий диск. Виртуальная память режет все виртуальное адресное пространство на страницы и отображает используемые страницы в физическую память. А если используемых страниц больше чем физической памяти, то мы выгружаем память на диск пока она нам снова не понадобится.

Плюсы:

- Освобождает программиста от необходимости вручную управлять загрузкой частей программы в память и согласовывать использование памяти с другими программами

- Освобождает программиста от необходимости вручную управлять загрузкой частей программы в память и согласовывать использование памяти с другими программами
- В многозадачных системах изолирует выполняющиеся программы друг от друга путем назначения им непересекающихся адресных пространств

Минусы:

- Переключение между приложениями может занять больше времени
- непредсказуемая длительность доступа к памяти(физически информация может лежать на магнитной ленте, потому что мы ее туда убрали до "востребования")

проблема фрагментации. Достоинства и недостатки

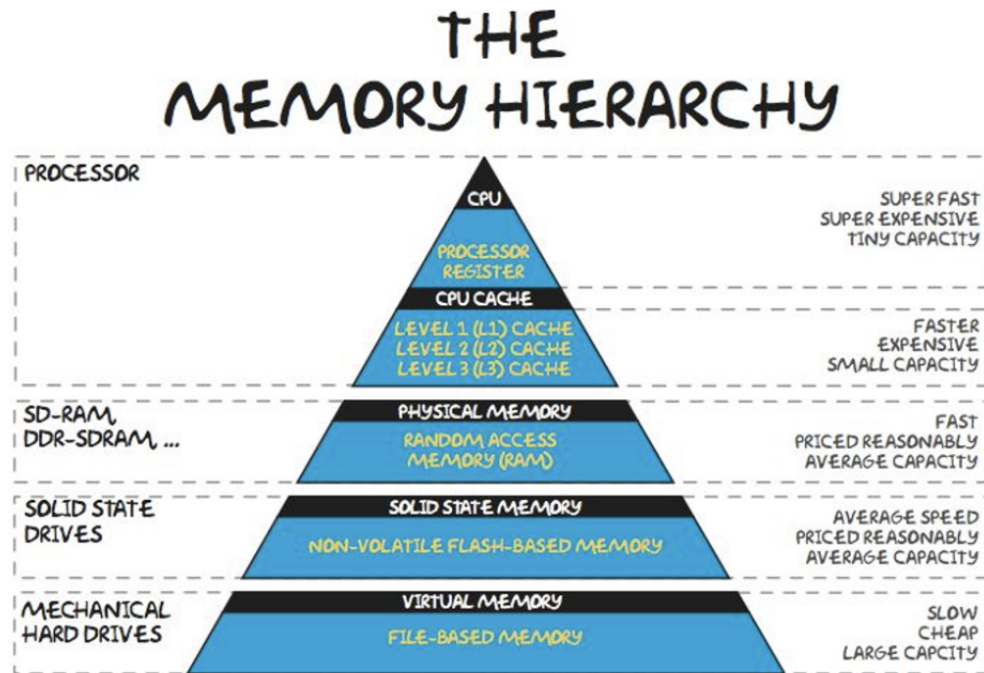
Управление памятью, основанное на динамических запросах на выделение или освобождение участков, приводит к тому, что память может оказаться разбитой на большое число перемежающихся занятых и свободных участков малых размеров (фрагментов). Это явление называемое **фрагментацией памяти**, приводит к весьма отрицательным последствиям: запросы на участки памяти не могут быть удовлетворены из-за малых размеров свободных участков, хотя суммарный размер свободной памяти достаточен для удовлетворения запросов

Фрагментация может привести к снижению производительности системы, поскольку операционная система тратит больше времени на поиск и чтение фрагментированных данных

Способы борьбы с фрагментацией памяти:

1. Расширение размера основной памяти
2. Упорядочение запросов на участки памяти по времени освобождения
3. Упорядочение запросов на участки памяти по размерам.
4. Включение алгоритмов сборки мусора

▼ 25) Иерархия памяти (явная и скрытая). Виды памяти. Особенности использования на практике. Устройство памяти с произвольным доступом. Устройство и принцип работы ROM, SRAM, DRAM ячеек.



- На верхнем уровне стоит процессор с его кешами, который представляет собой наиболее быстрый кусок памяти
- основная память
- SSD - твердотельные накопители, энергонезависимая память позволяющая хранить большой объём данных и обеспечивает произвольный доступ к ней;
- магнитные ленты - механические накопители

Явная иерархия памяти представляет собой структуру, в которой каждый уровень памяти имеет четкую идентификацию и доступен напрямую. Каждый уровень памяти имеет свой собственный адресный пространство и контроллер, который управляет доступом к памяти на этом уровне. Примерами явной иерархии памяти являются регистры процессора, кэш процессора и оперативная память

Скрытая иерархия памяти, с другой стороны, скрывает сложность управления памятью от пользователя. В этой модели, управление памятью осуществляется на более высоком уровне, и пользователь не видит или не управляет отдельными уровнями памяти. Примерами скрытой иерархии памяти являются виртуальная память

В основном сейчас везде скрытая иерархия. Мы обращаемся к виртуальной памяти, а оттуда уже понятно где лежит наша запись (в кэше, диске и тд, тут уже как повезет).

особенности использования на практике

Типов памяти условно два: с произвольным доступом - типа ссд, кэши и тд и последовательным - типа дисков и лент, есть еще гибридные (библиотека магнитных лент, где доступ к магнитной ленте реализуется произвольно, а доступ к данным в рамках ленты -- последовательно)

С практической точки зрения последовательный доступ в подавляющем большинстве случаев говорит не о невозможности произвольного доступа, а очень высокой длительности. К примеру жёсткий диск, где доступ к данным завязан на физическое позиционирование головки над магнитным диском, а значит последовательное считывание данных может быть очень быстрым, в то время как перемещение и поиск будет долгими за счёт ожидания перемещения головки и поворота диска.

Устройство памяти с произвольным доступом

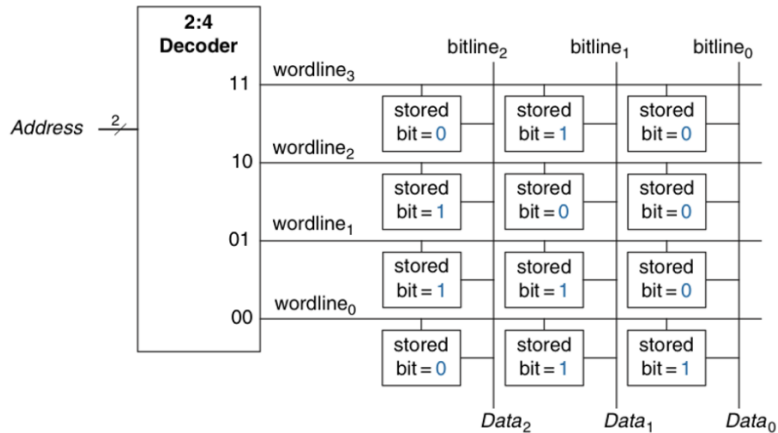


Figure 5.42 4 × 3 memory array

- **Address** — адрес ячейки памяти шириной 2 бита
- **Decoder** — дешифратор, активирует линию (1 из 4 шт.).
- **wordline_i** — линия, активирующая ячейки требуемого машинного слова.
- **bitline_i** — линия, на которую выставляется/читается значение бита определённой позиции.
- **stored_bit** — ячейка памяти

Здесь используется адрес размеров в 2 бита, соответственно, если размер будет n бит, то декодер будет иметь 2^n линий слова

Таблица реализаций ячеек памяти (просто посмотреть что дороже и быстрее):

Существует много вариантов реализации ячейки памяти. Типовые:

Memory Type	Transistors*Bit	Latency	Capacity	Cost/GB
Flip-flop (Триггер)	~ 20	20 ps	~ 1 – 5kB	a lot of \$
ROM	≤ 1	-	-	-
SRAM	6	1 – 10ns	~ 10kB – 10MB	~ \$1000
DRAM	1	80 ns	~ 10GB	~ \$10

В дальнейшем мы будем рассматривать эти маленькие ячейки

Флип-флоп - это базовый строительный блок цифровой логики, который может находиться в одном из двух состояний: 0 или 1. Флип-флоп может

переключаться между этими двумя состояниями, что позволяет ему использоваться для хранения информации и управления потоками данных в цифровых схемах

1) ROM (read only memory)

Read-Only Memory (ROM) - это тип неизменяемой памяти, который используется в компьютерах и других электронных устройствах. Данные, хранящиеся в ROM, не могут быть электронно изменены после производства устройства памяти. ROM полезен для хранения программного обеспечения, которое редко меняется во время работы системы, также известного как прошивочное ПО

2) SRAM (static random access memory)

состояние хранится на транзисторах, поэтому когда отключается электричество - пока информация, зато пока питание есть - данные на месте

6 транзисторов на ячейку памяти (два инвертора, на каждый по два транзистора и два транзистора на чтение данных), поэтому относительно быстро, не очень вместимо и довольно дорого

3) DRAM (dynamic random access memory)

DRAM хранит каждый бит данных в ячейке памяти, обычно состоящей из миниатюрного конденсатора и транзистор

Электрическая зарядка на конденсаторах постепенно утекает; без вмешательства данные на конденсаторе быстро исчезнут. Чтобы предотвратить это, DRAM требует внешней схемы обновления памяти, которая периодически перезаписывает данные в конденсаторах, восстанавливая их в исходное зарядное состояние. Наличие такого контроллера требует дополнительной логики, а также снижает скорость доступа к данным, так как доступ к ним в процессе регенерации становится невозможным.

В то же время, данный тип памяти требует наличия всего одного транзистора и конденсатора, что сильно увеличивает плотность ячеек памяти по сравнению с SRAM.

▼ **26) Механизм кеширования в компьютерных системах, принцип локальности. Функционирование кеш памяти процессора (чтение, запись). Виды кеш промахов. Механизм вытеснения (LRU, PLRU)**

кеш - промежуточный буфер с быстрым доступом, содержащий информацию, которая может быть запрошена с наибольшей вероятностью.

Программа в конкретный момент времени работает с малым объемом памяти причем достаточно концентрированной, т.е. адреса лежат относительно рядом. Работая с таким небольшим пучком данных достаточно частое явление, когда мы часто обращаемся к одному и тому же адресу памяти и когда мы обращаемся к адресам по соседству.

Временная локальность - часто стучимся в одну дверь

Пространственная локальность - если постучались в одну дверь, высока вероятность, что в скором времени постучимся в соседние

Кеши применяются на всех уровнях компьютерных систем: у браузера есть кеш для элементов с недавно посещенных страниц, у бэкенд/фронтенд приложений для часто повторяющихся запросов, список можно продолжать долго.

Большой кэш — не есть хорошо, потому что такое количество данных сложно предугадывать + скорее всего не будет так много переиспользуемых данных, а значит лишнее энергопотребление и площадь.

Данные между кэшем и памятью передаются блоками фиксированного размера, также называемыми **линиями кэша**

Чтение

Приходит адрес

1) находим соответствие в кэше → кэш-попадание (cache hit) → читаем из кэш-линии

2) нет соответствия → кэш-промах (cache miss) → запрашиваем данные ниже по иерархии (следующий кеш или память) → полученные данные нужно сохранить в кеше:

- а) Есть пустая линия → сохраняем в нее
- б) Все занято - выбираем линию, которую перезапишем (тут разные алгоритмы могут использоваться)
- в) тайминги получения данных - произвольные:) (многоуровневость кешей, блокировка памяти)

hit rate - как часто случается *cache hit*, основная характеристика кеша

Запись (при cache hit)

- а. **Немедленная запись** (*write-through*) - процессор что-то записывает в память, эта запись проходит сквозь кеш и сохраняется, уже далее двигает в память, таким образом кеш актуализируется сразу, даже быстрее чем основная память 😊. Может быть медленнее, сравнивая с тем, если бы кеша в этой цепочке не было.
- б. **Отложенная запись** (*write-back*) - записываем данные только в кэш, до памяти данные не доезжают, когда процессор сделал все, что хотел (прибавил/вычел/перевернул на 90 градусов - все это отразится только в кэше), данные доезжают до памяти. Такие строки называются "грязными". Возможно неконсистентное состояние кеша и памяти.

Если кэш будет заполнен и мы попытаемся прочитать по адресу, которого нет в кэше, произойдет два обращения в память:

1. строку в кэше, которую будем перезаписывать, если она грязная, записываем в память
2. читаем из основной памяти, ну и по ходу сохраняем в кэше, зря что ли грязнульку выгоняли

Запись (при cache miss)

- а. Запись с размещением - вытесняем кого-то из кеша, пишем в него, а потом в память (шаги сохранения записи в кеш аналогичны шагам при чтении с промахом)
- б. Запись без размещения - пишем в память, минуя кэш

Кэш-промахи (как себя чувствует процессор, когда они случаются)

1. При записи данных - задержка процессора минимальна, так как дожидаться завершения записи чаще всего не нужно
2. При чтении данных - средняя задержка, можно компенсировать параллелизмом уровня инструкций ака не нашлись данные для инструкции A, но нашлись для инструкции B
3. При чтении инструкций - тут совсем беда, процессору нужна инструкция, чтобы начать выполнять полезную работу

Механизмы вытеснения

LRU (Least Recently Used)

Выкидываем запись с самым ранним временем последнего доступа, можно реализовать банальным инкрементированием, будет геморрой с сравнением этих временных меток. Можно полечить, приложив подорожник в виде отдельной кэш линии, в которую будет попадать вытесненная запись перед тем, как быть выкинутой в мусорку (нормальных объяснений, почему это работает, не последовало: статистика говорит, что работает и все), либо запретив кеширование (мы точно знаем, что данные будут хаотично читаться без попаданий в кэш)

Pseudo-LRU

Оптимизируется LRU за счет бинарного дерева, где кэш-линии - это листья, а остальные узлы - либо 0, либо 1, которые показывают прошли ли через них в прошлый раз, т.е. из этих узлов можно построить путь к самой старой кэш линии (просто идем по ноликам). При спуске по дереву инвертируем значения.

▼ **27) Устройство кэш-памяти процессора. Ассоциативность кэш-памяти (полностью ассоциативная, прямое отображение, множественно-ассоциативный кэш). Принципы работы. Детальное описание принципов работы кэш-памяти с разными вариантами ассоциативности.**

кеш - промежуточный буфер с быстрым доступом, содержащий информацию, которая может быть запрошена с наибольшей вероятностью.

Кэш память процессора имеет аппаратную реализацию, поэтому, говоря о ней, оперируем реальными физическими элементами. В программной реализации кэш чаще всего представляет собой хэшмап.

Кэш должен поддерживать следующие свойства:

1. Поиск кэш линии по тегу
2. Вытеснение. Когда кэш заполнен, нужно выбрать линию, которую выкинем
3. Предзагрузка данных и инструкций. Кэш предугадывает следующие инструкции и команды и сохраняет их.
4. Взаимодействие с памятью
5. Синхронизация кэшей разного уровня

Заложенное противоречие

- a. Можно расширить аппаратно (например, добавить отдельный компаратор на каждую линию) и будем работать быстрее (1 такт!)
- b. Либо работаем медленнее (сравниваем каждую линию по очереди), но меньше элементов в реализации (N тактов)

Ассоциативность кэша

Это количество разных строк **кэша**, в которые можно отобразить одну строку памяти.

Имея на руках адрес нужно ему сопоставить линию кэша. Делается это с помощью тега линии.

Тег линии кэша - это часть кэш-линии, которая используется для определения того, соответствует ли данная кэш-линия тому адресу, который процессор пытается получить доступ. Тег линии кэша содержит

часть информации о том, где данные, хранящиеся в кэш-линии, были взяты из основной памяти

В качестве тега можно взять разные части адреса:

- 1) Полный адрес - много памяти, при сравнении дофига битов нужно сравнить ⇒ куча деталей в компараторе
- 2) Младшие биты - могут быть тупо не нужны из-за выравнивания (4КБ блоки в виртуальной памяти, например)
- 3) Средние биты - вот тут интересно получается, что они циклически повторяются, имеет место временная и пространственная локальность
- 4) Старшие биты - на практике достаточно уникальны, но на деле весь адрес нахрен не сдался никому

Полностью ассоциативный кэш (Fully Associative Cache)

В качестве тега берутся средние и старшие биты, младшие биты не нужны из-за деления на блоки, они служат просто как индекс для адресации внутри блока. Каждый блок памяти уникальным образом ассоциируется.

С точки зрения *hit rate* вариант идеальный, так как в каждую линию можно положить любой блок из памяти == более используемый адрес всегда будет добавлен в кэш, а менее используемый первым из нее будет выкинут

Но есть минусы в виде громоздкости аппаратной реализации: на каждую линию нужно поставить по компаратору. И это только чтение. Основная фишка - в 1 линию могут быть записаны ВСЕ блоки памяти

Главная особенность - в КАЖДОЙ линии может быть записан ЛЮБОЙ блок памяти (по картинке ниже это видно)

Кэш с прямым отображением (Direct Mapping)

Что сразу тут нужно понять: если в полном ассоциативном кэше мы компараторами искали подходящую линию, то в прямом отображении для

каждого адреса заранее известна линия, в которой она может потенциально лежать. Делается это с помощью средних битов.

В таком кэше все блоки с одинаковыми средними битами будут класться в одну кэш линию. По картинке ниже, в Cache Line 0 всегда будут блоки из списка: Block 0, Block 64, Block 256, так как средние биты у них одинаковые. Теги при этом, понятное дело, разные

По средним битам нашли линию, потом сверили старшие биты (тег), если нашли совпадение, то cache hit.

Плюсы на лицо - меньше тег, компаратор один, нету вытеснения, так как мы всегда знаем кэш-линию, куда нужно положить адрес

Минусы - представим серию обращений Block 0 _ Block 64 _ Block 0, все эти блоки будут лежать на одной линии, поэтому Block 64 перепишет Block 0, из-за чего в третьем обращении будет cache miss, хотя обращались мы к блоку Block 0 совсем недавно.

Множественно-ассоциативный кэш (N-Way Set Associative Cache)

Тут уже для адресов с одинаковыми средними битами выделяется не одна кэш линия, а несколько, это и есть канальность кэша (8 канальный кэш = 8 линий для блоков с одинаковыми средними битами)

Представим, что у нас 4-ех канальный кэш. Если мы запросим Block 0, то мы получим 4 кэш линии, в которых она может лежать. Теперь мы сравниваем теги нашего Block 0 с тегами этих линий, если в Direct Mapping был только один компаратор, то сейчас их 4, нашли совпадение - cache hit, иначе - cache miss.

Получается такая комбинация двух способов выше, используем Direct Mapping для получения линий и далее Fully Associative Cache для выбора из этих линий подходящей.

▼ 28) Иерархия кеш-памяти процессора. Причины множества уровней кеша. Разделённый/унифицированный, включающий/исключающий, частный/общий. Типовые уровни кеша в современных процессорах.

Так уж получилось, что данные используются совсем неравномерно: например, маленький набор адресов А мы можем дергать в 50% случаев, набор больше В - в 20%, еще больший набор С - в 5%. Если у нас память одна большая колбаса, то мы явно плюемся и ругаемся от того, что строки из А мы получаем с такой же скоростью, что и строки из С. Давайте сделаем малюсенький кэш, но со скоростью молнии для таких самых востребованных данных. Зажили здорово, теперь нас начинает коробить от того, что строку из В мы получаем с той же скоростью, что и строку из С, хотя обращаемся мы к ним гораздо чаще - отщипнули еще кэша, но, так как таких строк сильно больше чем строк из группы А, мы не можем так сильно разгуляться в аппаратной начинке, так как она дорогая, поэтому выбираем менее быструю, но с большим объемом. Примерно в таких рассуждениях и родились разные кэши, в конце концов статистически пришли к тому набору кэшей, который есть сейчас.

Сейчас стандартный набор процессора состоит из 3-ех кэшей - L1, L2, L3. Они расположились в порядке возрастания по времени доступа и объему.

Разделенный(Banked)/унифицированный кэш(Unified)

Разница в том, разделяем ли типы данных, которые хранятся в кэше.

Можем поделить данные по разным признакам:

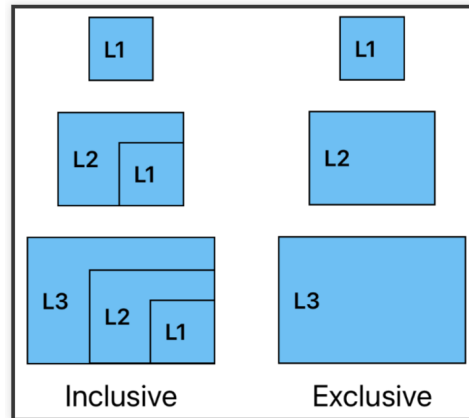
- 1) По типу (данные/инструкции/микрокод)
- 2) По процессорному ядру

Плюсы разделенного кэша (ну и недостатки унифицированного соответственно): повышаем скорость работы проца за счет увеличения каналов, т.е. одновременно забрали инструкцию и данные к ней; можно полезным образом заюзать возможность хранить разные типы в разных кэшах: например, для данных в кэше предиктим следующие значение одним образом, для команд - другим.

Минусы: сложность реализации (аппаратное разделение инструкций и данных; нужно продумать моменты, когда одно перетекает в другое; нужны команды, которые будут уметь работать с двумя кэшами)

Включающий/исключающий

Данные вышестоящих по иерархии источников данных дублируется в нижестоящий



Инклюзивный кэш приколел тем, что единица данных есть сразу на всех уровнях, из-за чего, например, выкинув данные из L1 кэша, то они автоматически добавятся в L2 кэш и не нужно проводить какую-то выгрузку из L1 в L2.

В инклюзивном: удалили что-то из кеша1 и забыли, это есть в кеше 2

в эксклюзивном кэше: удалили что-то из L1, надо теперь перенести это в L2

Также плюс инклюзивного кэша - это возможность прочитать данные из L1 другого ядра, не залезая в сам L1, взяв его, например, из L3

Из минусов инклюзивного кэша можно назвать дублирование данных - память кэшей съедается.

Эксклюзивный кэш - используем 100% памяти, но получаем сложный протокол взаимодействия между кэшами разных уровней, где мы перекидываем данные из одного кэша в другой.

Частный/общий

Сколько устройств/процессоров/... будут этим кэшем пользоваться.

Частный - 1 ядро имеет чисто свой кэш, только он в него ходит

- + скорость доступа
- дублирование данных на разных кешах

Общий - несколько ядер имеет доступ к кэшу

- + нет дублирования
- + меньше кэш промахов
- нужно синхронизировать доступ

Типовые уровни кэша

L0 (опциональный) - специальный кэш, скорее даже просто регистр, в который может записываться стек, int/float значения. Нужен для внутренних нужд процессора.

L1 - быстрый кэш, часть процессора, обычно разделенный

L2 - обычно часть процессора, шарится между ядрами, имеют заметно больший размер

L3 - самые большие, пришли к нам с приходом многоядерных процессоров, имеет функцию синхронизации данных в многоядерных системах, чтобы это не происходило через оперативку

L4 (экзотичный кэш) - оптимизация доступа к памяти в гигантских вычислительных системах

▼ 29) Когерентность кеш памяти. Возможные состояния кеш линий.

Механизмы обмена информацией между кешами: справочник, отслеживание и перехват. CAP теорема.

Когерентность - согласованность данных в кэшах на разных уровнях.

Есть проблема - параллельный доступ к данным (чтение/запись) множеством участников (ядра, DMA), кто-то может что-то записать в свой L1, что должно произойти с этим значением в L1 других процессоров и как это реализовать.

MOESI

Протокол поддержки когерентности кэш-линий. Предлагает состояния кэш-линий, на которые кэш должен как-то реагировать

- 1) **Модифицировано (Modified)** - Кэш-линия присутствует только в текущем кэше и её значение отличается от значения в основной памяти. Кэш обязан записать данные обратно в основную память в будущем, прежде чем разрешить любое другое чтение из основной памяти
- 2) **Владелец (Owned)** - Данные в кэше действительны и могут быть прочитаны только этим процессором, который ими владеет, и могут быть копии в других ядрах
- 3) **Эксклюзивное (Exclusive)** - Кэш-линия присутствует только в текущем кэше и её значение совпадает с значением в основной памяти
- 4) **Разделяемое (Shared)** - Эта кэш-линия может быть сохранена в других кэшах системы, совпадает со значением в основной памяти
- 5) **Неактуальное (Invalid)** - данные стали некорректными

Как кэши могут общаться

Справочник (Directory)

Общий справочник с информацией о разделяемых данных. Процессор перед чтением записи проверяет её в справочнике и обновляет в случае необходимости свою информацию. При изменении каталог может обновлять или аннулировать другие кэши с этой записью.

+ количество процессоров может быть большим, всем норм общаться с этим справочником

- задержка запроса к справочнику

- опять обращения куда-то == время == минус производительность.

Отслеживание (Snooping)

Кэш следит за шиной памяти, если видит, что какая-то запись изменена, то помечает линию как "неактуальную"

+ скорость

- ограниченная масштабируемость: процессор должен оповещать каждый процессор о том, что делает с памятью

Перехват (Snarfing)

То же, что Snooping, но измененная запись не просто помечается кэшем как "неактуальная", а целиком "перехватывается" и перезаписывается. Таким образом, такой механизм поддерживает максимальную согласованность. Но это очень дорого реализовывать.

CAP теорема

идея в том, что мы хотим иметь от хранилища следующие свойства:

- 1) Согласованность (Consistency) - каждое чтение получает последнюю запись
 - 2) Доступность (Availability) - каждый запрос получает ответ, даже если какой-то из узлов системы отвалился (тогда содержится последняя запись)
 - 3) Устойчивость к разбиению (Partition Tolerance) - система продолжает работать, несмотря на проблемы с коммуникацией между узлами сети
- эта теорема говорит, что все 3 свойства обеспечить невозможно, всегда нужно жертвовать хотя бы одним свойством

▼ 30) Закон Мура. Закон Деннарда. Закон Амадала. Их роль в развитии компьютерных систем. Источники роста производительности процессоров тогда и сегодня.

Закон Мура.

Закон Мура: количество транзисторов, размещаемых на кристалле интегральной схемы, удваивается каждые 2 года.

Закон Мура на самом деле никакой не закон, а просто наблюдение Гордона Мура.

С момента изобретения интегральной схемы в 1959 году количество транзисторов на микрочипах выросло в среднем в два раза каждый год. Гордон Мур это заметил, и сказал: Всё! Так и будет.

А в 1975 году он внес поправку, и сказал: " Нет, всё таки, каждые два года".

И вся индустрия подстроилась под такой темп.

И это, очень круто. Ведь чем меньше размер транзистора, тем меньше он потребляет тока. А чем больше количество транзисторов, тем выше вычислительная мощность. Причем зависимости прямо пропорциональные.

А значит, чем больше маленьких транзисторов получится разместить на чипе, тем лучше.

Однако в последнее время этот закон сталкивается с физическими и техническими ограничениями. С уменьшением размера транзисторов возникают проблемы, связанные с квантовыми эффектами и тепловым рассеиванием. Кроме того, стоимость производства современных микропроцессоров значительно возрастает. Все эти факторы вместе приводят к замедлению темпов удвоения количества транзисторов, что ставит под сомнение продолжительность соблюдения Закона Мура.

Закон Деннарда

Повышение скорости вычислений может достигаться за счет ускорения исполнения операций / преобразований данных, которое в свою очередь может достигаться за счёт **закона масштабирования Деннарда** – уменьшая размеры транзистора и повышая тактовую частоту процессора, мы можем легко повышать его производительность.

К сожалению, как и закон Мура, данный закон имеет свою область применимости и срок годности, так как наталкивается на:

- Ограничения современных технологий (сложность и стоимость производства).
- Физические ограничения на возможности уменьшения транзисторов (размеры атомов и эффекты связанные с уменьшением транзисторов).

Правило Деннарда закрепило уменьшение ширины проводника (техпроцесса) в качестве главного показателя прогресса. Но закон

масштабирования Деннарда перестал действовать примерно в 2006 году. Количество транзисторов в чипах продолжает увеличиваться, но этот факт **не дает значительного прироста** к производительности устройств.

Например, представители TSMC (производитель полупроводников) говорят, что переход с 7-нм техпроцесса на 5-нм **увеличит** тактовую частоту процессора всего на 15%.

Причиной замедления роста частоты, являются утечки токов, которые Деннард не учитывал в конце 70-х. При уменьшении размеров транзистора и повышении частоты ток начинает сильнее нагревать микросхему, что может вывести ее из строя. Поэтому производителям приходится балансировать выделяемую процессором мощность. В результате с 2006 года частота массовых чипов установилась на отметке в 4–5 ГГц.

Закон Амдала.

это утверждение о том, что производительность системы, состоящей из нескольких компонентов, зависит от производительности самого медленного компонента. Да, так и работает, если у нас очень быстрый процессор, но медленная память, то мы упираемся в память (это все нивелируется кэшами, регистрами и тд, но влияние медленных компонентов существует).

Ускорение программы с помощью параллельных вычислений на нескольких процессорах ограничено размером последовательной части программы. Например, если можно распараллелить 95% программы, то теоретически максимальное ускорение составит максимум 20х, невзирая на то, сколько процессоров используется.

**Их роль в развитии компьютерных систем.
Источники роста производительности
процессоров тогда и сегодня.**

Последние лет 50 направление и темп развития электроники определялись законом Мура.

3 типа влияния закона Мура на мир:

- соперничество разработчиков, т.е. борьба за первенство в создании более быстрого и более мощного процессора;
- разработка архитектуры вычислительных мощностей: обновление технологических алгоритмов происходит постоянно и регулярно (раз в 2 года);
- прогнозирование рынка.

Начиная с 2010 года, следование закону Мура перестало быть выгодным для разработчиков. На его соблюдение нужно тратить множество ресурсов: материалы, оборудование, увеличение штата и так далее.

Подводя итоги, источником роста производительности, является:

Повышение тактовой частоты.

Параллелизм.

Усложнение архитектуры

▼ 31) Проблема обеспечения реального времени в современных компьютерных системах. Влияние параллелизма уровня инструкций, языков программирования высокого уровня, многозадачности и организации памяти

Проблема обеспечения реального времени в современных компьютерных системах. Системы реального времени (Real-Time Systems, RTS) – это системы, в которых корректность работы не зависит только от логической корректности результатов, но и от времени, в течение которого эти результаты предоставляются. Другими словами, они должны гарантировать выполнение задач в строго определённые временные рамки.

Особенности и Вызовы

1. **Строгие временные ограничения:** В RTS задачи имеют чёткие сроки выполнения. Например, в автомобильных системах управления или

медицинских устройствах задержка может привести к серьёзным последствиям.

2. **Предсказуемость:** Помимо быстроедействия, важна предсказуемость поведения системы. Это значит, что система должна гарантировать выполнение задач в установленные сроки даже в условиях высокой нагрузки.
3. **Обработка прерываний:** Системы реального времени часто работают с оборудованием, которое может генерировать прерывания. Эффективная и быстрая обработка этих прерываний критична.
4. **Многозадачность и параллельная обработка:** Многозадачность требует особого подхода в распределении ресурсов и планировании задач, чтобы избежать конфликтов и удовлетворить временные ограничения.

Параллелизм уровня инструкций позволяет выполнять несколько инструкций одновременно. Это усложняет точное прогнозирование времени выполнения инструкций, что критично для систем реального времени. Необходимо учитывать данные и управляющие зависимости между инструкциями, которые могут привести к задержкам.

Языки программирования высокого уровня, такие как Python, Java, C#, предоставляют мощные абстракции и богатую стандартную библиотеку, упрощая процесс разработки. Но их использование в контексте систем реального времени имеет свои нюансы:

Сборка мусора: Автоматическое управление памятью может привести к непредсказуемым задержкам.

Уровень абстракции: Более высокий уровень абстракции может скрывать внутренние детали исполнения, что затрудняет оптимизацию производительности и анализ времени выполнения.

Многозадачность позволяет системе выполнять несколько задач одновременно. Это может повысить производительность системы, но

может также привести к задержкам в выполнении задач реального времени. Это связано с тем, что система должна распределять ресурсы между задачами, и задача реального времени может быть поставлена в очередь за другими задачами.

Организация памяти может повлиять на производительность системы и вероятность задержки в выполнении задач реального времени. Например, если система использует виртуальную память, то задача реального времени может быть вытеснена из оперативной памяти другой задачей. Это может привести к задержке в выполнении задачи реального времени.

Для обеспечения реального времени в современных компьютерных системах используются различные методы. К ним относятся:

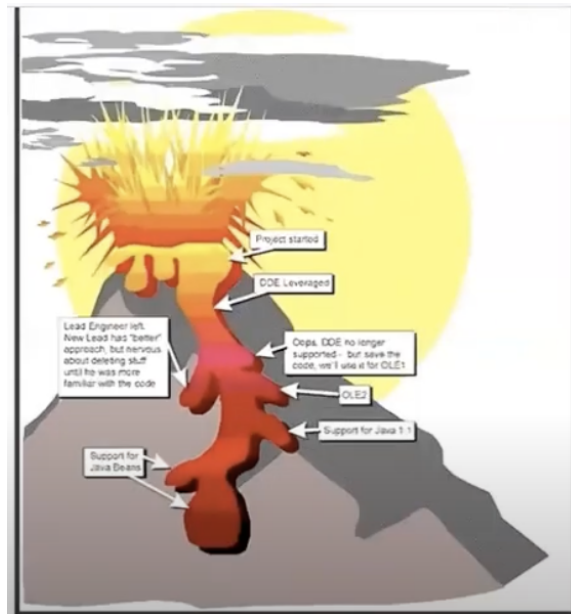
- Использование синхронизации. Синхронизация позволяет гарантировать, что задачи будут выполняться в определенном порядке.
- Использование приоритетов. Приоритеты позволяют системе отдавать приоритет задачам реального времени.
- Использование специальных аппаратных средств. Специальные аппаратные средства, такие как таймеры или процессоры реального времени, могут быть использованы для обеспечения реального времени.

▼ 32) Уровневая организация компьютерных систем. Элементы уровня организации. Уровневый архитектурный стиль. Примеры. Явление разделения на уровни и их смещения. Документирование инструментальных цепочек

Уровневая организация системы - естественный способ организации вычислительных систем. В любой системе (кроме самых примитивных) можно в том или ином виде увидеть уровневую организацию, можно

сказать что она является некой хронологией того, что исторически происходило с системой.

lava flow - со старта проекта используемые технологии менялись и приходилось постоянно что-то переписывать, т.е проект "тек" куда придется просто для того, чтоб продолжать его поддерживать

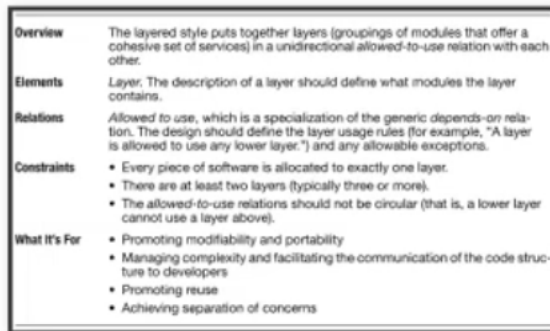


Одна из причин возникновения lava flow - смена людей: один ушел, другой пришел, ничего не понял и сделал по своему. Lava flow по сути не имеет никакой пользы для системы, является просто историческим побочным эффектом

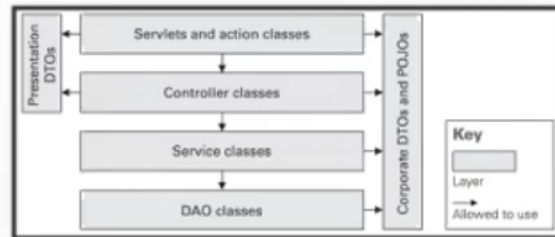
Уровневый архитектурный стиль

это базовый стиль проектирования, в котором мы разбиваем все доступные модули на независимые множества, где вышележащие множества могут обращаться к нижележащим.

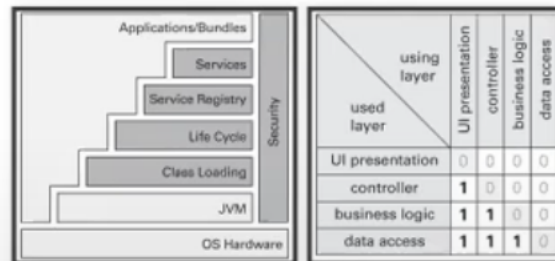
Пример: создали библиотеку А, начали делать библиотеку Б, используя уже готовую библиотеку А (но не наоборот).



(Layered Style)



- Изоляция, модифицируемость и переносимость
- Управление сложностью и структура кода разработчикам
- Разделения интересов



Особенностью уровневого стиля является то, что на самом деле мы почти никогда не можем выкинуть один из уровней, т.е. правильно спроектированная система работает так, что все уровни в ней необходимы.

Явный пример уровневой архитектуры: MVC модель

Сюда же можно вставить нашу любимую модель ОСИ:

1. Application Layer
2. Presentation Layer
3. Session Layer
4. Transport Layer
5. Network Layer
6. Data Link Layer
7. Physical Layer

Уровни организации вычислительного процесса

Любая современная система состоит из частей от фреймворка ЯП и ТЗ на исполнения до полупроводников и разрядов тока. Уровней между этими понятиями может быть огромное множество (чистый ЯП, машинный код, схемотехника и тд), и важно чтобы каждый уровень работал в

соответствии с поставленным ТЗ. Программируя что-то мы так или иначе изменяем один из уровней, но в благоприятном случае поведение других уровней тоже будет меняться в соответствии с тем, что мы написали.

Элементами уровней организации можно назвать то, что обеспечивает их работу:

1. Фреймворки, библиотеки, апишки
2. ЯП
3. Операционные системы
4. Системы команд и др.

Уровневая организация связана с двумя важными трендами в ВТ: disaggregation и смешение уровней.

Явление разделения на уровни (disaggregation)

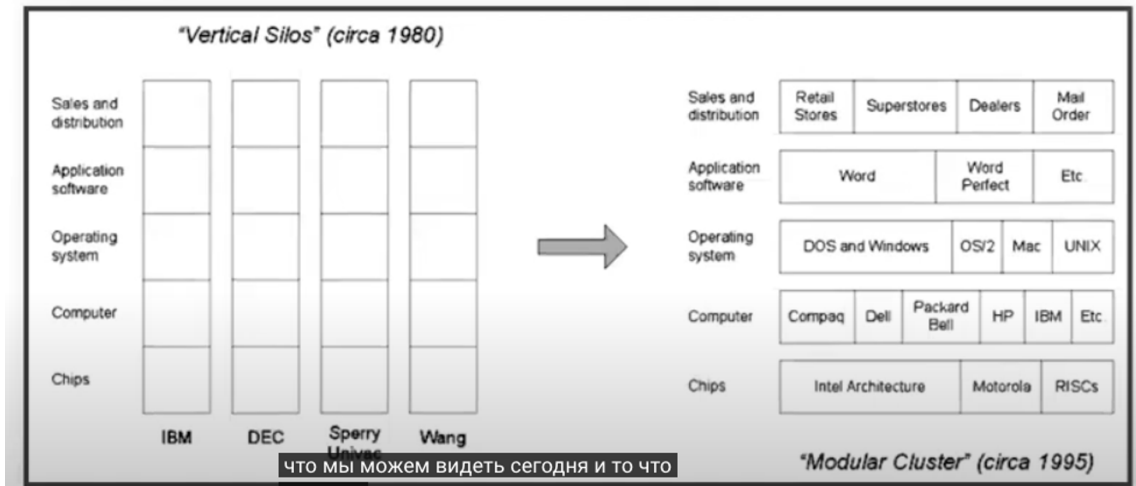
В начале зарождения сферы ИТ каждая компания внутри себя полностью выстраивала всю цепочку (все уровни) необходимые для работы: от чипов до ПО.

Такой подход оказался не оптимальным, т.к многим компаниям сложно выстроить полный цикл работы, но зато они могли превнести в сферу что-то свое узкое.

Начала происходить дисагрегация: компании начали разбиваться по признаку того, на чем они специализируются и стали взаимодействовать между собой. Конечно, это помогло добиться большей эффективности в работе каждой из них.

Недостаток:

в цельной системе уровни “изолированы” и слабо связаны между собой, что может привести к несовместимости

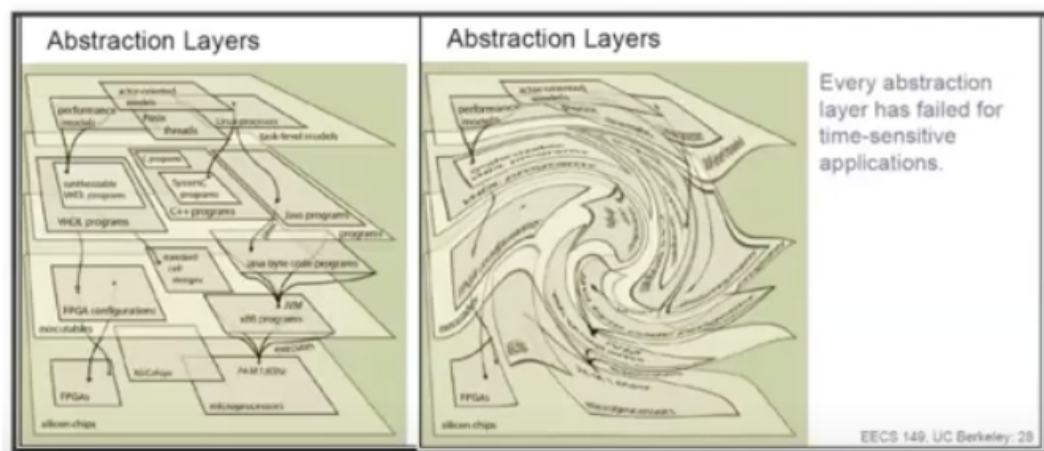


Смещение уровней. Интеграция

Это обратное дисагрегации, он "тянет" уровни системы к смешению и переплетению. Это может быть обусловлено многими факторами: требованиями безопасности, желанием оптимизации конкретного уровня за счет других и тд

Пример: при написании кода на СИ пишем ассемблерную вставку чтобы оптимизировать работу на конкретном процессоре

Минусы: теряется гибкость, продукт становится все более монолитным



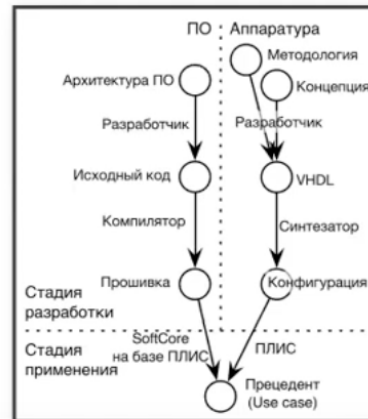
ну и последний пункт

Документирование инструментальных цепочек

При построении уровневой архитектуры остро стоит вопрос документирования, т.к зачастую просто разметить слои недостаточно, нужно точно показать взаимодействие элементов системы между собой. Один из способов это сделать - **граф актуализации**

Граф актуализации

1. Цепочка преобразований моделей выч. процесса через:
 - стадии жизненного цикла (Design → Dev → Use),
 - вычислительные платформы (High- → Low-level Langs → ISA → HW → Physic).
2. Ориентированный ациклический граф, вершины — модели, рёбра — трансляции/интерпретации.
3. Назначение: анализ инструментальных цепочек.



Нужен чтобы показать систему как набор представлений вычислительного процесса на всех уровнях, которые должны быть задействованы для того чтобы система корректно работала. По сути это roadmap работы системы с использованием ее составляющих частей

▼ 33) Особенности реализации структурного программирования в фон Неймановских процессорах. Работа с памятью, регистрами. Реализация процедур. Реентерабельность.

Структурное программирование - концепция разработки ПО, при котором используются не отдельные инструкции и go to, а целостные структурные блоки и их последовательности.

Признаки структурного высокоуровневого ЯП:

1. У кода есть структура и модульность, программа ≠ бесконечная портянка инструкций
2. Естественные элементы кода для архитектуры фон Неймана:

- а. условия
- б. циклы
- с. подпрограммы

По сути вышеперечисленные структуры и являются особенностями в архитектуре фон Неймана, о чем-то большем он в лекции не говорил

При попытке повысить уровень абстракции и использовать структуры появляются 2 проблемы: с регистрами и с реализацией подпрограмм, ниже про них подробнее

Распределение регистров. Выражения.

При использовании строк-выражений в высокоуровневых ЯП встает необходимость оперировать со всеми их аргументами, которых может быть произвольное количество. Но количество регистров в любом процессоре ограничено, поэтому появляется задача **распределения регистров** - отображение высокоуровневых значений и переменных на доступные регистры (в доступном количестве)

В этом процессе много нюансов, т.к если регистров не хватает, то мы начинаем буферизировать переменные и обращаться к памяти, что дорого. Сегодня этим эффективно занимаются компиляторы высокоуровневых ЯП.

Процедуры и подпрограммы

Архитектура фон Неймана на уровне процессора ничего не предлагает "из коробки" для реализации процедур и подпрограмм.

В целом существует 3 способа реализации подпрограмм:

1. через inline - просто каждый раз вставляем процедуру в виде последовательности инструкций.
Код раздувается, гибкость теряется, но при отсутствии других способов неплохой вариант
2. через goto - позволяет обращаться к определенной инструкции через условный переход, но при этом требует точку, куда нужно будет вернуться, а это тоже потеря в гибкости.

3. через call и return - самый оптимальный (имхо) способ, представлен сейчас языками, которые поддерживают вызов процедуры возврата (автоматически запоминают точку, куда надо вернуться)

Проблемы, связанные с вызовом процедур:

1. Кэширование. При вызове процедуры меняется область памяти, с которой мы активно работаем, а значит кэш-инструкции начинают не так эффективно работать и мы проседаем по производительности
2. Сброс регистров. Вызов процедуры заставляет компилятор думать о том, что происходит в регистрах, т.к подпрограмма может использовать регистры, используемые и в основном блоке, тем самым затерев их.
3. Переходы и конвейер. Не совсем понял идею, но вроде при jump'ах компилятор задумывается дольше, чем при последовательной работе

Реентерабельность

Если приводить функции и подпрограммы к современному виду, то необходимо поддержать возможность использовать их повторно.

2 подхода к реализации реентерабельности процедур через call и return:

1. Статическое выделение памяти для каждого входа в подпрограмму. Некая виртуализация вызова, дорого по памяти, но безопасно. Этим обычно занимается компилятор, если понимает что процедура может быть вызвана из двух точек одновременно
2. Современный вариант: автоматическая память через стек - каждый вызов подпрограммы выделяет место на стеке, которое будет считаться текущим "скоупом" и не сможет поломать параллельные вызовы из других мест. Т.е аллоцируются все необходимые данные на "глубоком" уровне стека

▼ 34) Уровни параллелизма. Параллелизм уровня бит. Низкоуровневый параллелизм. Параллелизм уровня инструкций. Параллелизм уровня задач. Примеры и особенности

Параллелизм - свойство систем, при котором несколько вычислений выполняются одновременно, и при этом, возможно, взаимодействуют

друг с другом. Так же, различные части блоков программ или задач могут выполняться не по порядку или в частичном порядке, при этом не влияя на конечный результат.

Это позволяет значительно повысить общую скорость выполнения в многопроцессорных и многоядерных системах.

Нужен тогда, когда:

- Когда нужно работать сразу над несколькими задачами
- Когда нужно повысить уровень утилизации ресурсов (не простаиваем, а работаем)
- Когда нужно повысить производительность компьютера (делаем больше дел за единицу времени)

Но как и при других способах увеличения скорости вычислений (уменьшая размеры транзисторов и увеличивая тактовую частоту), параллелизм имеет свои ограничения, которые описаны **законом**

Амдала: параллелизм не может увеличивать производительность бесконечно, когда-нибудь рост остановится.

Уровни параллелизма

1) параллелизм уровня бит - узлы комбинационной схемы работают параллельно

2) низкоуровневый параллелизм - означает параллельное выполнение нескольких операций на уровне аппаратного обеспечения. Это может включать в себя параллельную работу нескольких процессорных ядер, микропроцессорных устройств и других компонентов

3) параллелизм уровня инструкций - выполнение инструкций параллельно. Откуда взялся? выполнение команды - сложный и длительный процесс, а значит он может быть разделен на этапы и запущен на конвейере. Выполнение основной задачи (например, деления) может требовать много времени, а значит в процессоре будут присутствовать простаивающие элементы, которые можно загрузить другими задачами. Изменение порядка выполнения команд, зачастую, не изменяет их результата

примеры: Конвейеризация исполнения команд и **VLIW (Very Long Instruction Word)**: Архитектура, позволяющая выполнять несколько инструкций за один такт процессора.

4) параллелизм уровня задач - взяли например два вычисления не последовательно погнали вычислять, а параллельно

Это может быть достигнуто за счет использования параллельных схем, мультиплексоров и других цифровых логических устройств.

Параллельная работа важна во многих цифровых системах, поскольку она позволяет эффективно использовать ресурсы и может повысить общую производительность системы.

Примеры использования:

- **Многозадачность:** Современные операционные системы могут запускать и управлять множеством задач одновременно.
- **Распределённые Вычисления:** Использование нескольких компьютеров для параллельной обработки больших задач, например, в области искусственного интеллекта или при моделировании погоды.

▼ 35-36) Параллелизм уровня инструкций. Суперскалярные процессора. Особенности и принципы работы. Сравнение с VLIW и практика использования. Достоинства и недостатки. Барьеры памяти.

параллелизм уровня инструкций - это когда одна команда которую сейчас должен выполнить проц разбивается на этапы и каждый этап для разных инструкций выполняются параллельно.

Суперскалярные процессора - процессор, в котором существует параллельно несколько АЛУ, через которые мы можем одновременно считать и сумму и деление (сумму можно считать за такт, а вот деление нет).

В суперскалярном процессоре инструкция извлекается из потока инструкций (находящегося в памяти), определяется наличие или отсутствие зависимости инструкции по данным от других инструкций,

затем инструкция выполняется. Одновременно, в течение одного такта, может выполняться несколько независимых инструкций.

Станции резервирования эффективно выполняют переупорядочивание:

- Инструкции выполняются по мере доступности данных.
- Доступность данных редко совпадает с порядком инструкции в очереди.

Суперскалярные процессоры используют алгоритмы для предсказания исхода условных переходов, чтобы минимизировать задержки, связанные с ветвлениями в коде.

с точки зрения системы команд они идентичны простым процессорам и вся диспетчеризация происходит без ведома пользователя

Однако, увеличение производительности суперскалярных процессоров ограничивается сложностью поиска зависимостей между инструкциями и затратами времени на этот поиск, а также сложностью схемы. Кроме того высокая производительность сопровождается увеличением потребления энергии и тепловыделения

VLIW - Very Long Instruction Word

Основная идея VLIW заключается в том, что компилятор заранее определяет, какие инструкции могут выполняться параллельно, и упаковывает их в одну длинную инструкцию. разделением инструкций на параллель будет заниматься компилятор, а не процессор, тем самым разгружаем процессор и его устройство

Компилятор статически планирует и группирует независимые операции вместе в одну инструкцию VLIW. Это позволяет процессору одновременно выполнять несколько операций, что увеличивает производительность. Ещё тут нет механизма перестановки инструкций

достоинства

- 1. упрощение процессора, снижение энергопотребления**

2. **упрощение декодера. рост частоты:** т.к. уходит огромное кол-во схемотехники, которое должно работать на частоте процессора.
3. **компилятор имеет больше инфы о коде**, он лучше знает что параллельно. потенциал для оптимизации выше

недостатки

1. **сложность компилятора:** во многом из-за наличия процедур и динамических библиотек
2. **высокая нагрузка на каналы данных и регистровые файлы:** т.к. инструкции стали длиннее
3. **конфликты конвейера приводят к простоям всех узлов:** не можем получить входные данные, не можем выполнить инструкцию целиком → все простаивает. вытекает из предыдущего пункта
4. **ширина команды** – вопрос ограничения микроархитектуры. машинный код завязан на архитектуре процессора

Преимущества суперскалярных процессоров включают большую производительность на уровне инструкций и упрощенную аппаратную реализацию. Однако, они могут столкнуться с проблемами при выполнении инструкций, которые зависят друг от друга

у нас там есть доп буфер, в котором идет обратный реордеринг и если что-то что мы посчитали не пригодилось, то мы не откатываем изменение, а просто не записываем его в память, все решает процессор, программист отдыхает

В целом, выбор между суперскалярными и VLIW процессорами зависит от конкретной задачи и требований к производительности.

Суперскалярные процессоры могут быть более подходящими для общего назначения, в то время как VLIW процессоры могут быть более подходящими для специализированных приложений, таких как обработка сигналов и научные вычисления

Барьеры памяти

это инструкции, которые контролируют порядок выполнения операций чтения и записи в памяти. Они используются в многопроцессорных системах для обеспечения корректного выполнения операций чтения и записи в памяти

Есть несколько типов барьеров памяти:

- **Барьеры чтения:** Эти барьеры гарантируют, что все операции чтения, выполненные до барьера, будут завершены до любой операции чтения, выполненной после барьера. Они не влияют на операции записи
- **Барьеры записи:** Эти барьеры гарантируют, что все операции записи, выполненные до барьера, будут завершены до любой операции записи, выполненной после барьера. Они не влияют на операции чтения
- **Обобщенные барьеры доступа к памяти:** Эти барьеры гарантируют, что все операции чтения и записи, выполненные до барьера, будут завершены до любой операции чтения или записи, выполненной после барьера
- **Барьеры зависимости по данным:** Эти барьеры гарантируют, что данные, с которыми работает вторая операция, будут актуальными на момент выполнения этой операции.
- **Операции захвата блокировки (LOCK) и освобождения блокировки (UNLOCK):** Эти операции ведут себя как барьеры с односторонней проницаемостью. Они гарантируют, что все операции доступа к памяти, выполненные до операции захвата блокировки или освобождения блокировки, будут завершены до этого момента

▼ 37) Программируемые логические контроллеры (ПЛК). Область применения. Особенности. Языки программирования.

Программируемые логические контроллеры специализированные вычислительные системы, которые на производстве помогают решить разные проблемы управления

Например: на производстве есть конвейерная лента с шоколадками. Задача настроить чтобы правильно передавались шоколад, оборачивались в упаковку, управлять автоматикой, которая закрывает коробку и т.д.

Автоматизация при помощи релейных схем позволяла создать эту автоматику. Но современность показало, что это ненадежно т.к.

- плохо масштабируется
- сложно настраивать и поддерживать (шкаф с кучей проводов)
- сомнительная надежность сложных схем
- проблема соответствия документации и фактической реализации

придумали ПЛК, куда зашивался небольшой кусочек кода, который имитировал реле (очень надежно, меньше по площади и возможность относительно легко все поменять). По факту залили прошивку на чип, чип стоит где-то далеко и работает).

ПЛК могут быть встроены в оборудование и являться его неотъемлемой частью. Они автономны и реализуют функции контроля и управления. В системах противоаварийной защиты используется специальный тип локальных контроллеров, так как они отличаются высокой надежностью, живучестью и быстродействием

программирование ПЛК

ПЛК используют специализированные языки программирования, которые позволяют контролировать различные функции и операции, включая чтение и запись данных, управление двигателями и другими устройствами, обработку сигналов и многое другое. Эти языки программирования обычно имеют лёгкий доступ к манипулированию битами в машинных словах, в отличие от большинства высокоуровневых языков программирования современных компьютеров

Разрабатываются под конкретное устройство, с конкретными интерфейсами, у них специальные корпуса, приемники, разъемы и тп. Рассчитаны на работу в реальном времени.

▼ 38) Программируемые логические интегральные схемы (ПЛИС).

Область применения. Особенности. Устройство. Задачи высокоуровневого синтеза. Достоинства и недостатки.

ПЛИС – тип микрочипа, который может быть запрограммирован для выполнения различных логических функций. Они используются в различных областях, включая телекоммуникации, радиолокацию, компьютерные системы и многие другие. Функции могут быть изменены после производства, что делает ПЛИС гибким инструментом для проектирования цифровых систем. ПЛИС обычно используются в системах, где требуется гибкость и настраиваемость. Они могут быть использованы для создания различных цифровых систем, от простых цифровых фильтров до сложных систем обработки сигналов

Однако, несмотря на все преимущества, ПЛИС также имеют свои недостатки. Они обычно дороже (при штучном производстве точно, при серийном уже поприятнее) и занимают больше места на печатной плате, чем стандартные интегральные схемы. Кроме того, программирование ПЛИС может быть сложной задачей.

процесс работы:

1. пишется код на специальном ЯП: Verilog, VHDL
2. код переводится в набор булевых схем, которые позволяют описать логику
3. инструментарий помогает отобразить схему на производстве

как работает внутри?

чип представляет большую матрицу элементов. конфигурация ПЛИСа определяет какие ячейки с какими инструкциями связаны. отвечает за то какие входы/выходы к каким блокам подключены.

Логические блоки – самое главное (lookup tables) маски, которые реализуют правильную работу таблицы

Задачи высокоуровневого синтеза.

Для каждой схемы есть много вариантов реализации (т.е. даже если хотим один результат, то могут быть разные схемы)

Решает проблему: как сделать так, чтобы мы сначала записали алгоритм, а после решали в какую схему он превратится

Высокоуровневый синтез (High-Level Synthesis, HLS) — это процесс автоматического преобразования абстрактного описания алгоритмов (часто на языках высокого уровня, таких как C, C++, SystemC) в аппаратные описания. Этот процесс значительно упрощает проектирование интегральных микросхем (ИС), FPGA

Плюсы: быстро пишем код, переносимость между устройствами, оптимизация схемы.

Минусы: все в виде магии, мы не влияем на результаты синтеза, сложно найти человека, который умеет это делать хорошо, не всегда задуманная реализация соответствует действительности и проверить это бывает тяжело.

▼ 39) Классификация Флинна. Выделяемые классы и примеры машин этих классов. SIMT архитектуры

Классификация Флинна — это система, используемая для классификации архитектуры компьютеров на основе:

1. количества потоков инструкции,
2. количества потоков данных.

Достоинства: простая и понятная.

Недостатки:

1. Не применима к не фон Неймановским архитектурам.
2. Перегруженность класса MIMD.

Выделяемые классы процессов по Флинну:

1) **SISD (Single Instruction Stream / Single Data Stream)**: Одиночный поток команд и одиночный поток данных. К этому классу относятся

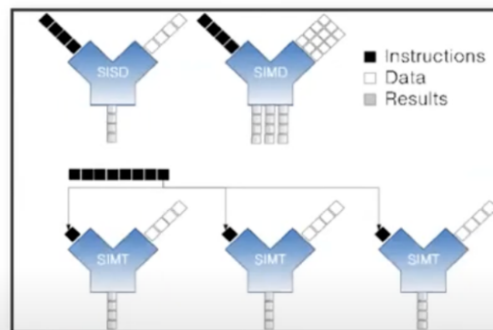
классические последовательные машины. Все команды обрабатываются последовательно друг за другом, и каждая команда инициирует одну операцию с одним потоком данных

Process unit последовательно взаимодействует с данными и инструкциями, никаких сложностей.

2) **SIMD (Single Instruction Stream / Multiple Data Stream)**: Одиночный поток команд и множественный поток данных. Это позволяет выполнять одну арифметическую операцию сразу над многими данными - элементами вектора. Применяется, например, в задачах графической обработки, в расчете матриц, математических задач и др

3)

SIMT (Single Instruction, Multiple Threads)



1. Расширение SIMD, позволяющее работать с оператором ветвления.
2. Широко применяется в современных GPU и GPGPU (CUDA).
3. Включает механизмы синхронизации потоков между собой, для достижения lockstep execution.

Является расширением SIMD и напрямую в классификацию НЕ ВХОДИТ.

У каждого процесса появляется личный counter и свое управление, поэтому такие процессоры имеют сложные механизмы синхронизации потоков между собой. Всё еще применяется в графической обработке, т.е в видеокартах

4) **MISD (Multiple Instruction Stream / Single Data Stream)**: Множественный поток команд и одиночный поток данных. В этом классе каждая команда может инициировать одну операцию с одним потоком данных. Класс отчасти искусственный и применяется редко. Пример из лекции:

использование для повышения уровня надежности системы (двоирование, т.е 2 потока работают идентично, и если это не так, то что-то сломалось)

5) • **MIMD (Multiple Instruction Stream / Multiple Data Stream):**

Множественный поток команд и множественный поток данных. В этом классе каждая команда может инициировать одну операцию с одним потоком данных. Самый популярный класс, почти все современные многопоточные процессоры построены на ней, классификация Флинна более никак не расширяет этот класс, поэтому подробнее они рассмотрены в классификации Дункана.

▼ **40) Классификация Дункана. Цели и задачи, классификация первого уровня. Синхронные архитектуры (векторные, SIMD, ассоциативные массивы, систолические). Принципы их работы и примеры.**

Классификация Дункана была призвана актуализировать и расширить классификацию параллельных процессоров Флинна

Задачи:

1. Абстрагироваться от низкоуровневого параллелизма.
2. Переиспользовать элементы таксономии Флинна (1966).
3. Актуализировать и включить новинки (1990).

Но она получилась сложной, плохо структурированной (схема классификации треш) и искусственной (некоторые классы впихнули чтобы впихнуть)

Основные критерии классификации Дункана (классификация первого уровня):

1. Синхронные архитектуры (критерий синхронности)

Параллельные архитектуры с единым механизмом управления (глобальные часы, центральный блок управления и т.п.). Различаются механизмами синхронизации.

2. MIMD архитектуры (multiple instruction pool, multiple data pool)

Параллельные архитектуры, которые могут исполнять независимые потоки инструкций. Потоки инструкций исполняются автономно,

требуется динамическая синхронизация. Различаются структурой организации памяти.

Т.е процессоры не зависят друг от друга, но в конце динамически синхронизируются.

Акцент делается на то, каким образом реализовано обращение к данным, поэтому выделено в отдельный класс

3. **MIMD парадигма** (MIMD paradigm architectures)

Архитектура MIMD процессора определяется структурой и МоС (модель вычислений). МоС определяет характер потоков данных, принципы взаимодействия и синхронизации. Различаются МоС. (хз что имеется в виду, но суть такова, что этот класс выделяется из-за специфичного построения системы на программном уровне, этот класс можно назвать "че-то около MIMD, но есть нюанс")

Синхронные архитектуры

Синхронные архитектуры - это тип архитектур, в которых все операции выполняются последовательно и в определенном порядке. В синхронных системах каждая операция начинается только после завершения предыдущей операции. Это обеспечивает строгое порядковое соответствие между операциями и делает эти системы простыми для понимания и анализа

Основное отличие: есть единая точка (юнит), которая дирижирует всей остальной системой (обработчик событий какой-то)

1. **Векторные архитектуры**

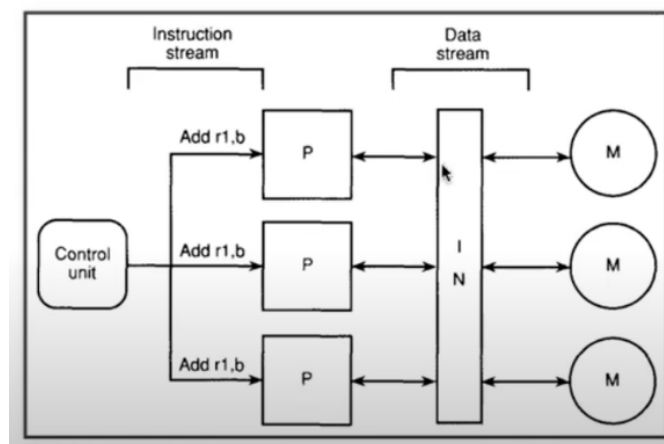
Позволяют обрабатывать не только скалярные величины, но и наборы однотипных данных (вектора и кортежи) "скопом", т.е нет отдельной инструкции на каждый элемент вектора. **Устройство:** обычно для векторных вычислений присутствует свое АЛУ, которое выполняет роль "конвейера" элементов вектора/кортежа. Достоинство такой архитектуры заключается в экономии на чтении и декодировании инструкций, т.е действий выполняется столько же, а инструкций считано меньше.

2. SIMD архитектуры

SIMD (Single Instruction, Multiple Data) - это тип параллельного процессинга, который используется в архитектуре компьютера. В SIMD-архитектуре одна и та же инструкция применяется к нескольким наборам данных одновременно. Это позволяет ускорить выполнение некоторых типов операций, таких как обработка изображений или аудио, где одна и та же операция должна быть выполнена на большом количестве данных

Подразделяются на несколько подгрупп, вот они слева направо:

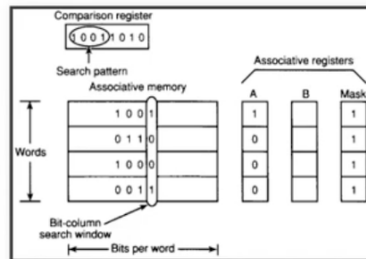
SIMD. Процессорные массивы



Являются эквивалентом SIMD в классификации Флинна, т.е имеют многопоточное обращение к памяти, но инструкции считываются синхронно.

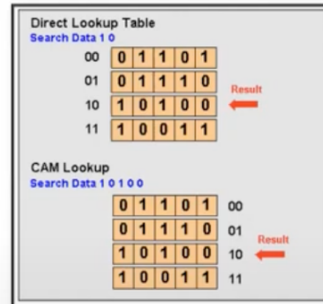
SIMD. Ассоциативный массив

(Content-Addressable Parallel Processor, CAPP)



Позволяют быстро искать информацию в массивах данных.
Используется в сетевых устройствах.

Процессор основанный на ассоциативной памяти (Content-addressable memory, CAM)



Это способ построения вычислительных систем, который строится не вокруг ядер и не вокруг комбинационных схем, а вокруг ассоциативной памяти, т.е вместо прямого обращения к памяти по адресу используется так называемый CAM (смотри правый рисунок), который работает в другую сторону: по желаемым данным возвращается номер ячейки памяти.

Такой способ хорош когда нужно быстро понять есть ли в массиве данных желаемое значение. Используется в кэш-памяти, в сетевых устройствах.

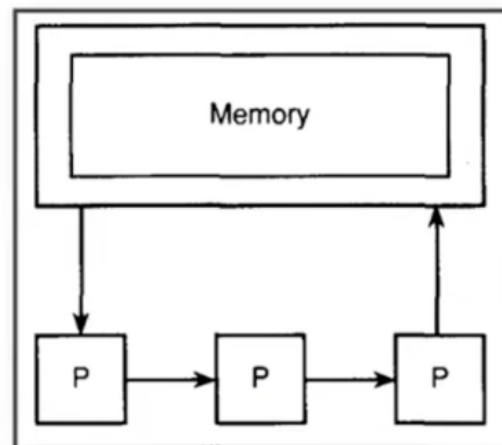
Sync. Системные архитектуры

Конвейерные мультимикропроцессоры:

1. данные ритмично поступают из памяти,
2. проходят через сеть процессоров,
3. и возвращаются в память.

Синхронизация — через глобальные часы, такт.

Процессора объединены локальными связями, и каждый цикл процессора выполняют короткие неизменные операции.



Суть заключается в том, что есть набор процессоров, соединенных друг с другом, они в конвейерном формате обрабатывают данные и передают следующему. Ритм поступления данных задается общим тактом системы.

Может дать выигрыш при нагруженных математических вычислениях (матрицы и тп), но является сложным в реализации: надо поддерживать системный такт, синхронизацию, для оптимальной работы задачи должны быть разбиты на жесткие рамки длительности, иначе смысла в такой архитектуре нет.

▼ 41) Классификация Дункана. Цели и задачи, классификация первого уровня. MIMD архитектуры (распределённая и разделяемая память). Примеры и принципы их работы.

Классификация Дункана была призвана актуализировать и расширить классификацию параллельных процессоров Флинна

Задачи:

1. Абстрагироваться от низкоуровневого параллелизма.
2. Переиспользовать элементы таксономии Флинна (1966).

3. Актуализировать и включить новинки (1990).

Но она получилась сложной, плохо структурированной (схема классификации треш) и искусственной (некоторые классы впихнули чтобы впихнуть)

Основные критерии классификации Дункана (классификация первого уровня):

1. Синхронные архитектуры (критерий синхронности)

Параллельные архитектуры с единым механизмом управления (глобальные часы, центральный блок управления и т.п.). Различаются механизмами синхронизации.

2. MIMD архитектуры (multiple instruction pool, multiple data pool)

Параллельные архитектуры, которые могут исполнять независимые потоки инструкций. Потоки инструкций исполняются автономно, требуется динамическая синхронизация. Различаются структурой организации памяти.

Т.е процессоры не зависят друг от друга, но в конце динамически синхронизируются.

Акцент делается на то, каким образом реализовано обращение к данным, поэтому выделено в отдельный класс

3. MIMD парадигма (MIMD paradigm architectures)

Архитектура MIMD процессора определяется структурой и MoC. MoC определяет характер потоков данных, принципы взаимодействия и синхронизации. Различаются MoC. (хз что имеется в виду, но суть такова, что этот класс выделяется из-за специфичного построения системы на программном уровне, этот класс можно назвать "че-то около MIMD, но есть нюанс")

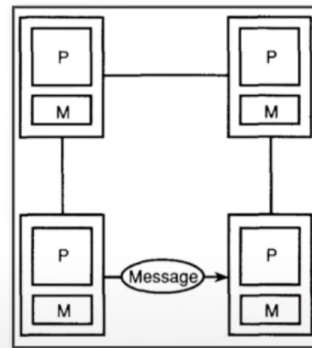
MIMD архитектуры

Выделяется 2 подтипа:

1. С распределенной памятью

MIMD. Распределённая память

1. Координация совместной работы через прямое взаимодействие процессоров.
2. Данные/сигналы явно передаются между процессорами (message passing).
3. Мотивация: обеспечить масштабируемость многопроцессорной архитектуры и доступ к локальным данным процессора.



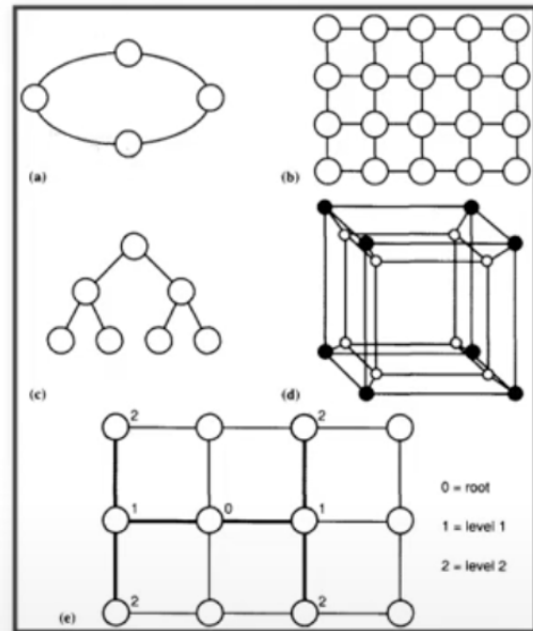
У каждого процессора есть своя локальная память, т.е свой набор регистров, кэшей и прочего, он с ней работает независимо от других процессоров. Это позволяет каждому процессору работать независимо, но также требует механизмов для обмена данными и координации между процессорами

Для микросервисная архитектура низкого уровня, недостатки и достоинства такие же: сложно разрабатывать и соблюдать параллелизм оптимально, но зато легко масштабировать и получать данные обособленно из каждого процессора

Отдельным пунктом здесь можно выделить топологию таких систем (как процессоры связаны между собой)

В зависимости от задач может быть выбран тот или иной шаблон построения:

1. **Кольцевая** топология + [хорды]. Мало процессоров, передача данных не доминирует.
2. **Ячеистая** / сетка. Соединены соседи + [диагональные] + [кольцевые]. Топологическое соответствие матричным алг.
3. **Дерево**. Алгоритмы поиска и сортировки, обработка изображений, потоковые и редуccionные машины.
4. **Гиперкуб**. Коммуникационный диаметр $\log_2 N$.
5. **Реконфигурируемые**. Фиксируем сеть конфигурацией, а не маршрутом.

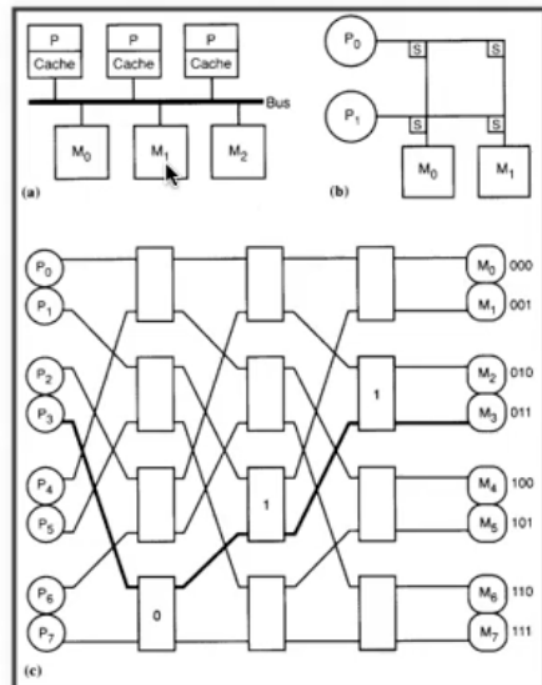


используется в автоматизированное производство, моделирование, а также коммуникатор связей (устройство, которое используется для обмена информацией между различными системами или компонентами в рамках распределенной системы)

2. С разделяемой памятью

MIMD. Разделяемая память

1. Координация совместной работы через общую память.
2. Ключевая проблема — синхронизация доступа к памяти, согласование кешей.
3. Примеры подходов:
 - (a) шинное соединение;
 - (b) перекрестная шина 2x2;
 - (c) сеть на кристалле, запрос маршрутизируется от P к M.
4. Широко распространены, так как строятся привычных процессорах и с привычным программированием.



Процессоры в данном классе “никак” не связаны между собой. У всех процессоров есть доступ ко всей памяти, особенности лишь в том, как этот доступ реализован (см. картинки)

Все наши современные телефончики и компютеры так или иначе реализуют данную архитектуру.

▼ 42) Классификация Дункана. Цели и задачи, классификация первого уровня. MIMD парадигмы (MIMD/SIMD, потоки данных, редукционные, wavefront) . Примеры и принципы их работы.

Классификация Дункана была призвана актуализировать и расширить классификацию параллельных процессоров Флинна

Задачи:

1. Абстрагироваться от низкоуровневого параллелизма - параллельная работа чего-то аппаратного, например нескольких ядер.
2. Переиспользовать элементы таксономии Флинна (1966).

3. Актуализировать и включить новинки (1990).

Но она получилась сложной, плохо структурированной (схема классификации треш) и искусственной (некоторые классы впихнули чтобы впихнуть)

Основные критерии классификации Дункана (классификация первого уровня):

1. Синхронные архитектуры (критерий синхронности)

Параллельные архитектуры с единым механизмом управления (глобальные часы, центральный блок управления и т.п.). Различаются механизмами синхронизации.

2. MIMD архитектуры (multiple instruction pool, multiple data pool)

Параллельные архитектуры, которые могут исполнять независимые потоки инструкций. Потоки инструкций исполняются автономно, требуется динамическая синхронизация. Различаются структурой организации памяти.

Т.е процессоры не зависят друг от друга, но в конце динамически синхронизируются.

Тут важно именно то каким образом реализовано обращение к данным, поэтому выделено в отдельный класс

3. MIMD парадигма (MIMD paradigm architectures)

смотрим на логику управления системой, смотрим на MoC (модель вычислений) и принципы взаимодействия данных и синхронизации.

MIMD парадигма

1. Гибридные архитектуры MIMD/SIMD

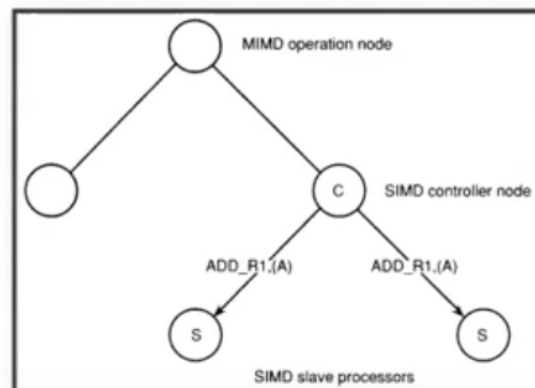
Фишка в использовании древовидной структуры, каждый процессор работает "в однопоточном" режиме, но каждое его действие - делегирование задач нижележащим нодам

Гибридным назван потому, что корневые ноды работают в SIMD формате

SIMD (Single Instruction, Multiple Data) - это тип параллельного процессинга, который используется в архитектуре компьютера. В SIMD-архитектуре одна и та же инструкция применяется к нескольким наборам данных одновременно. Это позволяет ускорить выполнение некоторых типов операций, таких как обработка изображений или аудио, где одна и та же операция должна быть выполнена на большом количестве данных

MIMD/SIMD

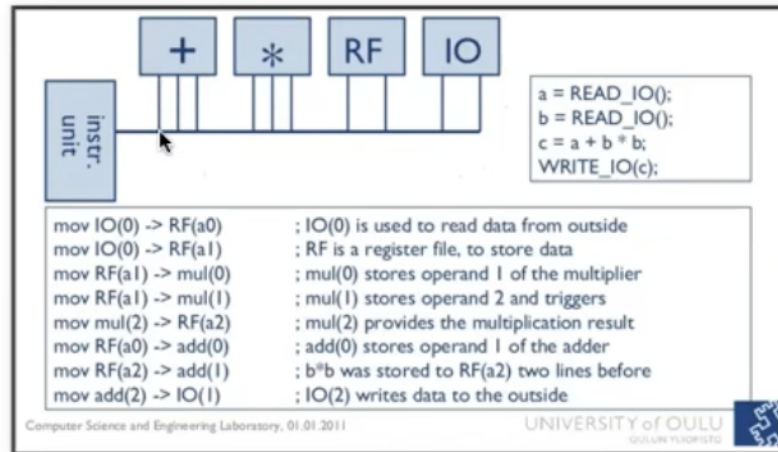
1. Гибридные архитектуры, где управление MIMD (малые задачи) осуществляется в стиле SIMD (крупные задачи).
2. Отношение ведущий — ведомый (master — slave) может быть представлено деревом:
 - корень — работает как SIMD и распределяет задания;
 - листья — работают как MIMD с локальной памятью.



Пример: диспетчер виртуальной машины, который распределяет запущенные процессы по разным физическим ядрам

Авторский пример пенского: система, которая не выполняет действия напрямую, а пересылает данные между модулями которые умеют что-то делать.

MIMD/SIMD. Transport Triggered Architecture (TTA) Processor



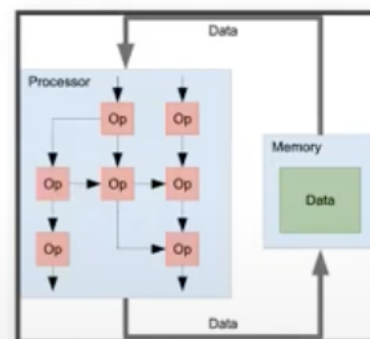
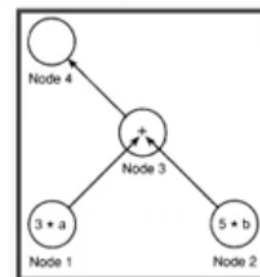
2. Потокковая архитектура (Dataflow)

Вычисления представляются в виде ориентированного графа, где каждая вершина это какая-то операция

Потокковая архитектура (Dataflow)

Потоковые архитектуры

возможность выполнения и исполнение инструкций определяется исключительно наличием входных аргументов для инструкций. Порядок выполнения инструкций непредсказуем.



(реконфигурируемая топология)

1. Алгоритм представляется в виде графа вычислений.
2. Отсутствует память и канал доступа как "бутылочное горлышко".
3. Проблема — передача данных между узлами.

Является некой инверсией предыдущего примера: каждая нода выполняет свое действие и передает результат вышестоящей, а та в свою очередь ждет ВСЕ ноды, которые ей нужны для исполнения.

Достоинство: не требует общего тактового управления, все ноды работают обособленно и непредсказуемо (динамическая синхронизация). Высокий уровень параллелизма, несмотря на то что некоторые ноды блокируются нижележащими

Проблема: сложно организовать оптимальную пересылку данных между нодами, чтобы некоторые не стояли в долгом ожидании

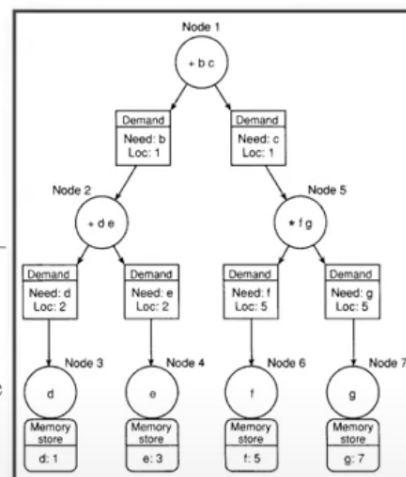
3. Редукционная архитектура (Reduction)

Редукционная архитектура (Reduction)

Редукционные архитектуры (ориентированные запрос)

инструкции разрешаются к выполнению, когда их результаты требуются в качестве операндов для уже разрешенных инструкций.

1. Разрабатывались для параллелизма и ФП.
2. Выполнение заключается в выборке инструкций и их замене на результат.
3. Проблемы: ссылочная прозрачность, разворачивание вычислительного графа, лень.



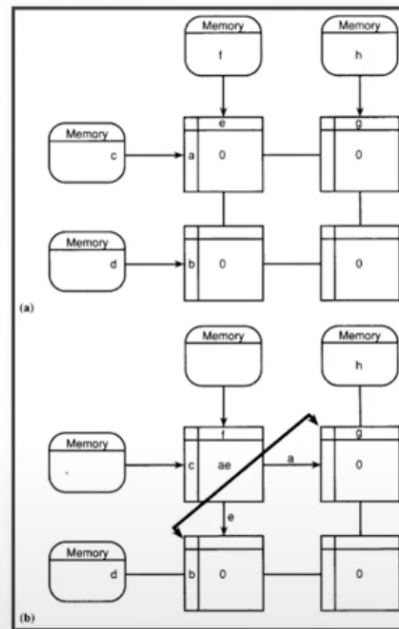
При поступлении запроса к системе происходит анализ "Что нужно, чтобы посчитать?". Далее динамически определяется в какие зависимости он уходит и начинают выполняться только необходимые действия.

Используется в функциональном программировании как один из способов параллелизма, но реализовать такое достаточно сложно, т.к необходимо обеспечить эффективность системы наравне с обеспечением памяти, ведь нужно предусмотреть ноду для каждого случая входных данных

4. Wavefront

Wavefront

1. Комбинация систалической архитектуры с асинхронным потоковым исполнением.
2. Управление происходит динамически, за счёт распространения сигнала по сети.
3. Синхронизация и режим работы выстраивается динамически.
4. Проще в программировании. Лучше масштабируется.



Принцип похож на синхронную систолическую архитектуру, но все ядра независимо работают с памятью и распространяют свои сигналы.

По факту динамический аналог последовательного исполнения
Примеров он не привел(